

COSMIC–ASCEC

*Automated Configurational and Conformational Sampling
with Topological Screening of Molecular Clusters*

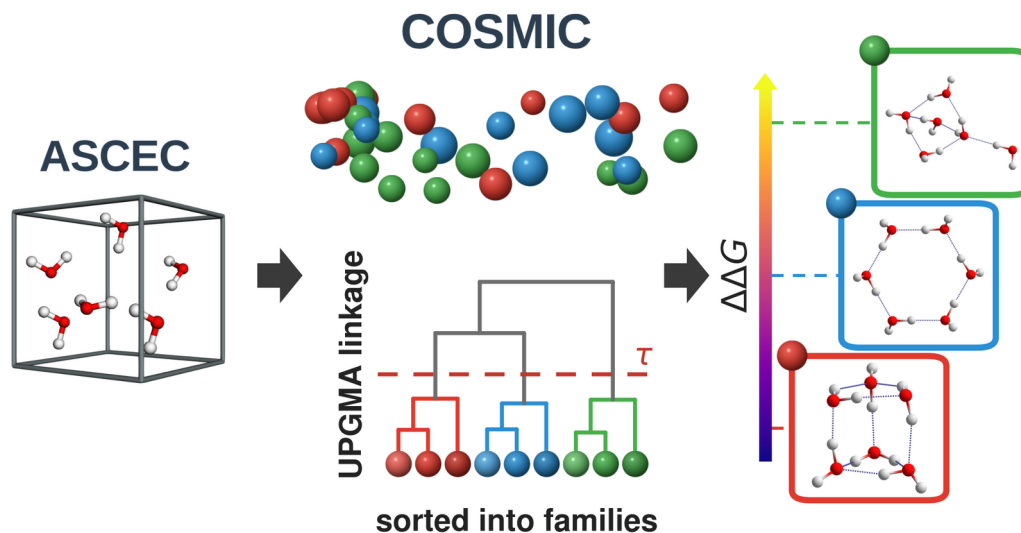
User Manual

Manuel Gómez[†] – Sara Gómez[‡] – Albeiro Restrepo[†]

Química Física Teórica

[†]*Instituto de Química, Universidad de Antioquia UdeA, Medellín, Colombia*

[‡]*Universidad Nacional de Colombia, Departamento de Química, Bogotá, Colombia*



June 2026

Contents

1	About COSMIC-ASCEC	4
1.1	Search Modes	4
1.2	Distribution	5
1.3	How to Cite	5
2	Installation	5
2.1	System Recommendations	5
2.2	Required Software	6
2.2.1	Core Python Dependencies	6
2.2.2	Quantum Chemistry Backends	6
2.3	Step by Step Installation	6
2.3.1	Prerequisites	6
2.3.2	Automatic Installation	7
2.3.3	Installing on a Cluster	8
2.3.4	Manual Installation with Conda	10
2.4	Quick ORCA 6.1.1 Installation for Linux	12
3	Web Input Generator	14
3.1	Features	14
3.2	Accessing the Tool	14
3.3	Smart Mode	14
3.4	Before You Download	15
4	Quick Start: Glycolaldehyde and Water	16
5	Water Hexamer	19
5.1	Molecular Coordinates Generation	19
5.2	Mode Selection	19
5.3	Simulation Box	19
5.4	Annealing Parameters	20
5.5	Geometry Optimization and Refinement	21
5.6	Energy Refinement and COSMIC Screening	21
5.7	Pipeline Review	22
5.8	Running and Monitoring the Job	22
5.9	Results	24
6	Theoretical Background	27
6.1	Simulated Annealing Algorithm	27
6.2	Monte Carlo Moves	28
6.3	Topological Clustering (COSMIC)	28
6.3.1	The Physicochemical Feature Vector	29
6.3.2	Weighted COSMIC Metric	29
6.3.3	Hierarchical Two Step Clustering	29
6.3.4	Feature Vector Technicalities	30
6.3.5	A Dynamic Feature Vector	30
6.3.6	Quality Control	31
6.3.7	Automatic Threshold Selection	31
7	Input File Format	32
7.1	Advanced Generation	32
7.2	Block 1: Simulation Parameters and Coordinates	32
7.3	Block 2: The Automated Protocol and Embedded Templates	34
7.3.1	Input Templates	36
7.3.2	Advanced: the Full xTB Flag Set	36
8	Box Size Suggestion	38

8.1	How V_{eff} is Estimated	39
8.2	Choosing the Packing Fraction	39
9	Standalone Functions: Understanding Each Stage	40
9.1	Annealing	40
9.2	Geometry Optimization	42
9.3	Sorting the Results	43
9.4	COSMIC Clustering	44
9.5	Geometry Refinement	45
9.6	Standalone Mode Scope	46
10	Workflow Mode	47
10.1	Pipeline Logic	47
10.2	Review Runs: Rehearsing an Expensive Protocol	48
10.2.1	Review Run versus Full Run: the Li_5 Cluster	50
10.3	Worked Example: Formic Acid Dimer	50
10.4	Execution Control and Resumption	54
10.4.1	Stage Delimiters	54
10.4.2	Resuming the Workflow	54
10.4.3	Restarting from a Stage	56
10.5	Failure Recovery	56
10.5.1	Retry: Launch Failures	56
10.5.2	Redo: Convergence and Topology	56
11	Output Files	57
11.1	Print Levels: miniprint and maxprint	58
11.2	Annealing Output	58
11.2.1	Main Log File	59
11.2.2	Reproducibility and Metadata	60
11.2.3	Simulation Generated Data	60
11.3	Calculation Output	63
11.4	COSMIC Output	64
11.4.1	Threshold Diagnostic	64
11.4.2	Dendrograms	65
11.4.3	Clustering Summary Log	66
11.4.4	Error Handling and Diagnostics	67
11.4.5	Generated Cluster Data	68
11.4.6	Boltzmann Distribution	69
12	Results and Validation	69
12.1	Screening Plain Coordinates	70
12.2	Clustering Methodologies: Topology vs. Geometry	71
12.2.1	Invariance to Atom Indexing	71
12.2.2	Comparison with Existing Methods	72
12.2.3	Quantifying a Visual Redundancy	73
12.2.4	When to Use RMSD	73
13	Command Reference	73
13.1	Cheat Sheet	74
13.2	Automated Workflow (Protocol)	75
13.3	Protocol Stage Flags	75
13.4	COSMIC Analysis Module	75
14	AI Acknowledgement and Reproducibility	76
15	Glossary	76
16	References	77

Part I

Getting Started

1 About COSMIC-ASCEC

COSMIC-ASCEC performs an automated configurational and conformational search on the potential energy surface (PES) of atomic and molecular clusters. A system in thermal equilibrium populates many minima at once, so the aim is the whole set of structures that contributes to its chemistry, not only the global minimum. A single input file, built with the web generator, is all it takes: the program runs every stage without manual intervention and returns the distinct minima with their relative energies and Boltzmann populations.

With COSMIC-ASCEC you can explore a new cluster from scratch, choosing how far to take the result, from a quick semiempirical map to verified minima refined at any level of theory available in ORCA or xTB (Section 1.1). COSMIC also works on its own, reducing any ensemble, such as plain .xyz files or the output of other programs, to its unique motifs, and every stage can be run separately to inspect or repeat a step.

The search runs in three stages:

1. **Sampling, ASCEC** (*Annealing Simulado Con Energía Cuántica*). Simulated annealing moves, rotates and deforms the molecules inside a box, with energies from a quantum chemistry program, and yields hundreds of candidates.
2. **Screening, COSMIC** (COnfigurational Similarity via Motif Identification Clustering). The preoptimized candidates are grouped into families by their physicochemical descriptors (energies, dipole, hydrogen bonds, among others), with no atom numbering or superposition, and one representative per family, a **motif**, is kept.
3. **Refinement**. Only the motifs are reoptimized at a higher level of theory, with frequencies that confirm true minima, and screened again.

1.1 Search Modes

Every run is one of three modes. They differ in how far the pipeline continues after the first clustering pass, and therefore in what the result supports.

Preliminary runs annealing, geometry optimization and one COSMIC screening, entirely at a low cost semiempirical level. It maps the surface in minutes, which makes it the natural first step with any new system. Because no frequencies are computed, the structures it returns are candidates rather than confirmed minima.

Rigorous adds a geometry refinement at a higher level of theory with a frequency calculation, followed by a second COSMIC screening. The frequencies confirm each structure as a true minimum and supply the thermal corrections, so this mode delivers verified minima with relative free energies and Boltzmann populations. This is the mode for results you intend to publish.

Ultimate adds a high level single point energy refinement on the refined geometries, for example DLPNO-CCSD(T), and a final COSMIC screening. The geometries are those of the rigorous mode, what improves is the accuracy of the relative energies that order them.

Cost rises steeply across the three. A preliminary run of a small system takes minutes, while rigorous and ultimate runs of the same system take hours to days, almost all of it in the refinement stages. Start preliminary, confirm the setup, then rerun in a higher mode.

Review is not a fourth mode but a modifier that applies to any of the three. A review run (`--review`) executes every stage of the chosen protocol, yet carries only the lowest energy structure produced by the annealing, the putative global minimum, into the expensive stages. It is the cheap way to confirm that a rigorous or ultimate protocol is correct before committing to it, and is described in Section 10.2.

1.2 Distribution

- **Repository:** <https://github.com/manuel2gl/qft-cosmic-ascec>
- **Web input generator:** <https://manuel2gl.github.io/qft-cosmic-ascec/>
- **License:** GNU General Public License (see the LICENSE file).
- **Platforms:** developed and tested on Linux (Ubuntu, Debian, Mint). On Windows 11 only preliminary runs have been tested, with xTB 6.7.1.

1.3 How to Cite

If you use COSMIC-ASCEC in published work, please cite:

- **Paper:** M. Gómez, S. Gómez, A. Restrepo, *[title, journal and DOI to be added; manuscript under consideration]*.
- **Reference data:** inputs and outputs of the benchmark runs, Zenodo, DOI [10.5281/zenodo.20723683](https://doi.org/10.5281/zenodo.20723683).

The `docs/` folder of the repository collects some of the relevant literature, the earlier ASCEC studies of water, methanol, formic acid, gold and other clusters, as a starting point for related work and for citing the method.

2 Installation

2.1 System Recommendations

Although Python scripts are lightweight, quantum chemistry backends are resource intensive:

- **RAM:** Minimum 8GB (16GB+ recommended for DFT calculations).
- **CPU:** Multicore processor (ASCEC can parallelize ORCA jobs).
- **Storage:** SSD recommended. Optimization trajectories can generate large text files.
- **OS:** Linux (Ubuntu/Mint) is the primary supported environment. Windows is supported for preliminary runs using xTB.

2.2 Required Software

2.2.1 Core Python Dependencies

The core functionality is based on standard Python scientific libraries and other software.

Package	Purpose
NumPy	High performance array operations and linear algebra.
SciPy	Optimization routines and statistical functions.
scikit-learn	Algorithms for hierarchical clustering and data preprocessing.
Matplotlib	Generation of energy plots and dendrograms.
cclib	Parsing of legacy ORCA ($\leq 5.0.x$) logs.
OPI	Parsing of modern ORCA (≥ 6.1) outputs.
OpenBabel	File generation and format conversion.
xTB	Highly efficient semiempirical quantum chemical framework.

Table 1: Software required by the ASCEC and COSMIC modules.

***Compatibility Note:** The **COSMIC** module automatically selects the appropriate parser based on the output file version. **cclib** is used for ORCA $\leq 5.0.x$, while **OPI** is required for ORCA 6.1+. *Note: ORCA 6.0 is not supported by either parser; please use v5.0.x or upgrade to v6.1+.* At least one parser must be installed.

2.2.2 Quantum Chemistry Backends

The workflow requires an external electronic structure package to perform QM calculations.

ORCA: ORCA [1] is free for academic use and supports a broad range of high level ab initio and DFT methods. It is fully integrated with both the ASCEC and COSMIC modules. See Section 2.4 for installation instructions.

xTB: xTB [2] is the default backend for both the annealing and preliminary preoptimization stages due to its speed. It is installed automatically from **conda-forge** by **install.sh** and **win_install.bat**.

2.3 Step by Step Installation

2.3.1 Prerequisites

git is required. The installer makes your copy of COSMIC-ASCEC a real git checkout, which is what lets you update it later with **git pull** and see your own edits with **git status**. Install it before anything else.

On Linux, use your distribution’s package manager:

```
sudo apt install git      # Debian, Ubuntu, Mint
sudo dnf install git      # Fedora, RHEL
sudo pacman -S git        # Arch
```

On Windows, install Git for Windows from <https://git-scm.com/download/win>.

Then confirm it is on your PATH:

```
git --version
```

No administrator rights (for example on a shared cluster)? Run the installer anyway: it falls back to installing git from conda-forge, which needs no root. That copy lives inside the conda environment, so it is only on your PATH after conda activate, which is why a system wide git is preferable whenever you can get one.

2.3.2 Automatic Installation

We provide a shell script that automates the entire process. It installs Miniconda (if not present), sets up the dependencies (xTB, OpenBabel, cclib, OPI), and configures the necessary aliases. The aliases point directly to the environment's Python binary, so no manual conda activate is needed to run ascec or cosmic.

Download the script, run it, delete the download and reload your shell, with one command. It installs into a Python 3.11 environment named py11:

```
cd "$HOME" && wget \
  https://raw.githubusercontent.com/manuel2gl/qft-cosmic-ascec/main/install.sh \
  && bash install.sh && rm -f install.sh && source ~/.bashrc
```

If wget is not available, swap the download for curl:

```
cd "$HOME" && curl -fLO \
  https://raw.githubusercontent.com/manuel2gl/qft-cosmic-ascec/main/install.sh \
  && bash install.sh && rm -f install.sh && source ~/.bashrc
```

The leading cd "\$HOME" keeps the download out of any existing checkout, so the rm can only ever remove the copy you just fetched. Confirm the install:

```
ascec --version
```

Installing into base instead

If you would rather not have a separate environment, install the latest Miniconda, downgrade Python within base, then edit the flag at the top of the script and run it again.

```
micro install.sh
```

```
INSTALL_PY11=FALSE
```

Save the file (Ctrl+S) and exit (Ctrl+Q).

Uninstall (Linux)

To reverse the installation without touching your Conda base, run the provided uninstaller.

```
bash uninstall.sh
```

This removes the `py11` environment, the `~/software/ascec04` directory, and the `ascec/cosmic` aliases from `.bashrc`. Conda itself is left intact.

Windows Installation

On Windows the same workflow is provided as a `.bat` file (a hybrid batch/PowerShell script). Install Git for Windows first (Section 2.3.1), then download `win_install.bat` from the repository and either double click it or run it from a Command Prompt. The script installs Miniconda if absent, creates the `py11` environment, installs all dependencies (including xTB 6.7.1), and registers `ascec` and `cosmic` as launchers on your user PATH. If Windows blocks execution, right click the file and select *Run as administrator*.

From a Command Prompt, with one command:

```
cd /d "%USERPROFILE%" && curl -fLO ^
https://raw.githubusercontent.com/manuel2gl/qft-cosmic-ascec/main/win_install.bat
^
&& win_install.bat && del win_install.bat
```

Then open a new Command Prompt so the updated PATH is picked up, and check:

```
ascec --version
```

Uninstall (Windows)

Double click `win_uninstall.bat` (or run from Command Prompt) to remove the `py11` environment, the source directory, and the launchers. Miniconda is left untouched.

```
win_uninstall.bat
```

2.3.3 Installing on a Cluster

On a shared machine you usually cannot install software as root, and the site already provides conda as a module. The installer is built for this: it never writes into a shared conda prefix, and every path it uses can be redirected from the command line. Load the site's conda and run the installer as usual:

```
module load anaconda3          # or whatever your site calls it
bash install.sh
```

The installer locates conda by asking it, with `conda info --base`, so any conda already on your PATH is found automatically whatever its layout: Miniconda, Anaconda, miniforge, a module or

a container image. Only when no conda is on `PATH` does it look at the usual prefixes, and only if that also finds nothing does it install Miniconda itself. Set `CONDA_ROOT` solely to override that choice, for instance to select one of several installed condas:

```
CONDA_ROOT=/apps/anaconda3 bash install.sh
```

The environment is then created in your own `envs` directory (normally `~/.conda/envs`) whenever the shared prefix is read only, so nothing you do affects other users. No step needs `sudo`: if git is missing the installer pulls it from `conda-forge` into the environment.

Home directories with a quota. Send both the software and any Miniconda the script installs to a larger filesystem:

```
TARGET_DIR=/scratch/$USER/ascec04 \
  CONDA_INSTALL_DIR=/scratch/$USER/miniconda3 \
  bash install.sh
```

Older site conda. If the module's conda has neither mamba nor the libmamba solver, skip the probing and use the classic solver directly:

```
CONDA_SOLVER=classic CONDA_ROOT="$(conda info --base)" bash install.sh
```

Every knob is an environment variable read at the top of `install.sh`; the defaults apply when you set nothing.

Variable	Purpose
<code>CONDA_ROOT</code>	Select a specific conda prefix. Rarely needed: any conda on <code>PATH</code> is detected automatically.
<code>CONDA_INSTALL_DIR</code>	Where a Miniconda installed <i>by the script</i> should land. Default <code>~/miniconda3</code> .
<code>TARGET_DIR</code>	Where the source is installed. Default <code>~/software/ascec04</code> .
<code>ENV_NAME</code>	Name of the conda environment. Default <code>py11</code> .
<code>PY_VERSION</code>	Python version for that environment. Default <code>3.11</code> .
<code>CONDA_SOLVER</code>	<code>auto</code> , <code>mamba</code> , <code>libmamba</code> or <code>classic</code> .
<code>INSTALL_MAMBA</code>	Whether mamba may be added to the base environment. Never touches a base the script does not own.
<code>MINICONDA_URL</code>	Installer to download; change it on <code>aarch64</code> or <code>ppc64le</code> , or to pin a release.
<code>REPO_URL</code> , <code>REPO_BRANCH</code>	Install a fork or a test branch.
<code>ALLOW_TARBALL</code>	Permit the plain directory install on a machine that truly cannot have git.

Table 2: Environment variables accepted by `install.sh`.

Login shells read `~/.bash_profile` rather than `~/.bashrc`, so the installer writes its alias block to whichever of `.bashrc`, `.zshrc` and `.bash_profile` exist. That is what makes `ascec` resolve inside a batch job.

2.3.4 Manual Installation with Conda

Use this route when you want to see every step, or when the automatic installer cannot run. It does exactly what `install.sh` does.

I. Get the source. Clone the repository. Later, `git pull` in the same directory updates it.

```
mkdir -p "$HOME/software" && git clone \
  https://github.com/manuel2gl/qft-cosmic-ascec.git "$HOME/software/ascec04" \
  && cd "$HOME/software/ascec04" && pwd && ls
```

II. Install Miniconda (skip if you already have conda). Answer `yes` when it offers to initialize conda.

```
cd "$HOME" && wget \
  https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh \
  && bash Miniconda3-latest-Linux-x86_64.sh && source ~/.bashrc
```

III. Create the environment and install the dependencies.

```
conda create -n py11 python=3.11 -y \
  && conda install -n py11 -c conda-forge numpy scipy matplotlib \
  scikit-learn cclib openbabel 'xtb>=6.7' -y \
  && conda run -n py11 pip install orca-pi
```

Both ORCA parsers can coexist: cclib reads ORCA $\leq 5.0.x$ and OPI reads 6.1+. OPI is installed with pip even inside conda.

IV. Add the aliases. Pointing them at the environment's own Python binary means you never need `conda activate` to run either program. This writes the two lines for you, using your real conda path:

```
CB="$(conda info --base)" && printf '%s\n' \
  "# COSMIC ASCEC aliases" \
  "alias ascec='$CB/envs/py11/bin/python $HOME/software/ascec04/ascec-v04.py'" \
  "alias cosmic='$CB/envs/py11/bin/python $HOME/software/ascec04/cosmic-v01.py'" \
  \
  >> ~/.bashrc && source ~/.bashrc
```

V. Verify. No environment activation needed:

```
ascec --version && cosmic --version && obabel -V
```

```
conda run -n py11 python -c \
  "import numpy, scipy, matplotlib, sklearn, cclib, opi; print('All packages OK')"
```

If you installed ORCA as well (Section 2.4), check it too with `orca -v`. The help screen confirms the command is wired up.

```

$ ascec -h

usage: ascec [OPTIONS] COMMAND [ARGUMENTS]

ASCEC - Annealing Simulado con Energía Cuántica
(Simulated Annealing with Quantum Energy)
Configurational and conformational sampling via Monte Carlo with quantum
mechanical evaluations

options:
-h, --help      show this help message and exit
-v             Increase verbosity level (use -v, -v2, -v3, etc.)
--standard     Use standard Metropolis criterion instead of modified
--nosum       Skip summary file generation during sort
--nobox       Disable generation of box-visualization XYZ files
--maxprint     Keep all files (default: miniprint, clean up at end)
-V, --version  Display version information and exit

COMMANDS:
Run an annealing job (the COMMAND is the .asc input file itself):
ascec input.asc          Run simulated annealing on input.asc
[...]

```

The command list continues. The full set is in the command reference, Section 13.4.

```
$ ascec -V
```

```

=====

*****
*       A S C E C       *
*****

          √≈≠==≠
√≈≠==≠√  √≈≠==≠√  ÷++=
÷++÷      ÷++÷      =++=
=++=      =++=      ≠==≠ ÷++=
=++=      =++=      =xx÷=≠=÷++=
=++=      =++=      ≠xx= ÷++=
=++=      =++=      =+÷   =++=
≠x+÷      ÷+≠≠      =+÷   =++=
=xx÷      =xx=      ≠xx= ÷++÷
          ≠=====≠  ≠÷÷÷=≠≠=xx+÷-
          ≠==≠      ≠==≠ ≠==≠      ≠==≠

          √≈≠==≠
          ÷++=
          =++=
          ≠==≠ ÷++=
          =xx÷=≠=÷++=
          ≠xx= ÷++=
          =+÷   =++=
          =+÷   =++=
          =+- - -xx+xx÷x
          ≠x-x
          ≠÷÷÷=≠≠=xx+÷-
          ≠=====≠√
          ≈=====

          ≠==≠      ≠==≠      ≈=====

          University of Antioquia - Medellín - Colombia

          Annealing Simulado Con Energía Cuántica

          * ASCEC-v04: Feb 2026 *

          Química Física Teórica - QFT

=====

```

2.4 Quick ORCA 6.1.1 Installation for Linux

xTB is installed with COSMIC-ASCEC and is enough for preliminary runs. ORCA is needed for the DFT and wavefunction refinements of rigorous and ultimate runs. The steps below install ORCA 6.1.1 on Linux.

1. Check for AVX2 support

AVX2 builds are faster. If this prints 0, use the plain build in the next step.

```
grep -c avx2 /proc/cpuinfo
```

2. Download the archives

Register on the ORCA forum and download the ORCA 6.1.1 shared library build (AVX2 or plain) and OpenMPI 4.1.8. ORCA is linked against one specific OpenMPI release, so the versions must match. These steps use the x86-64 .tar.xz archive; the Installer, Arm64 and RISC-V versions are not covered.

- ORCA download portal: <https://orcaforum.kofo.mpg.de/app.php/portal>
- OpenMPI releases: <https://www.open-mpi.org/>

3. Move the archives to ~/software

If your browser saves elsewhere, change \$HOME/Downloads to that folder. Both archives should appear in the listing.

```
mkdir -p "$HOME/software" && mv "$HOME/Downloads"/orca_6_1_1_linux*.tar.xz \
"$HOME/Downloads"/openmpi-4.1.8.tar.gz "$HOME/software/" \
&& cd "$HOME/software" && ls -lh
```

4. Build OpenMPI

This compiles OpenMPI into \$HOME/software/openmpi-4.1.8 using every available core, and takes several minutes. The build folder, openmpi-4.1.8-src, is removed at the end, since only the installed folder is used.

```
cd "$HOME/software" && mkdir -p openmpi-4.1.8-src \
&& tar -xf openmpi-4.1.8.tar.gz -C openmpi-4.1.8-src --strip-components=1 \
&& cd openmpi-4.1.8-src \
&& ./configure --prefix="$HOME/software/openmpi-4.1.8" \
&& make -j"$(nproc)" && make install \
&& cd "$HOME/software" && rm -rf openmpi-4.1.8-src
```

Check the build, which should report 4.1.8:

```
"$HOME/software/openmpi-4.1.8/bin/mpirun" --version
```


3 Web Input Generator

The COSMIC-ASCEC web input generator builds the `.asc` input file in the browser, with nothing to install.

3.1 Features

- **PubChem Integration:** converts a compound name, SMILES, CAS number or InChI (e.g. `water`, `64-17-5`, `CCO`) into 3D coordinates through the PUG REST API [3].
Compounds search: <https://pubchem.ncbi.nlm.nih.gov/compound/>
- **3D Visualization:** previews the molecules and the box boundaries with 3Dmol.js [4], and a **Visual Test** mimics the random rigid body moves of the annealing to check the box size.
- **Protocol Builder:** assembles the multistage workflow (`opt`, `cosmic`, `ref`, `eref`) for the chosen mode and summarizes it before download.

3.2 Accessing the Tool

1. **URL:** visit <https://manuel2gl.github.io/qft-cosmic-ascec/>
2. **Terminal:** open the same page in your browser with

```
ascec input
```

3. **Local server:** serve the copy shipped with your installation from your own machine, at `http://localhost:8080` (give another port as `ascec input local 8081`). The PubChem search still needs an internet connection.

```
ascec input local
```

3.3 Smart Mode

The generator has two interfaces, selectable from the top navigation bar. **Smart Mode**, the default, is a guided workflow that produces a valid input with minimal effort. **Advanced Generation** exposes every field of the input file, and is described in Part II (Section 7.1). Both write the same `.asc` file. The typical Smart Mode sequence is:

1. **Build the system.** Fetch each species from PubChem in the *Molecule* $\rightarrow$ *XYZ Converter* and set its number of copies.
2. **Set the simulation box.** Give the box length directly, or enable *auto* to compute it from a target packing percentage, and check it with the Visual Test.
3. **Choose the mode and parameters.** Select a preliminary, rigorous or ultimate run (Section 1.1), optionally as a review run. A guided form fills sensible defaults for the annealing, optimization, refinement and COSMIC settings, so only the key choices need adjusting.
4. **Review and download.** The pipeline summary shows every stage before you download the input file.

The screenshot displays the COSMIC-ASCEC web application interface. At the top, navigation links include 'Installation', 'Smart Mode' (selected), 'Advanced Generation', 'User Manual', and 'GitHub'. A header bar contains the text 'Automated configurational sampling and topological screening of molecular clusters', 'Química Física Teórica', and 'Universidad de Antioquia, Colombia'.

The main content area is divided into three primary sections:

- Simulation Box:** Features a 'Box Length (Å)' input set to 6.0, a 'Dense' checkbox, and a 'Visual Test' button. A 3D visualization shows a molecular cluster within a cubic box. A 'Box Suggestion — adjust as needed' message is present. On the right, 'Added molecules (3)' are listed: 'Water (2)' and 'Glycolaldeh... (1)'. Below the box, there are buttons for 'Refresh', 'Download XYZ', 'Download Box XYZ', and an 'Enter simulation name' field.
- Molecule → XYZ Converter:** Includes a text input for 'Enter any PubChem identifier (name, SMILES, CAS, InChI) to generate 3D coordinates.' Below this is a list of molecule buttons: Water, Ethanol, Formic Acid, Alanine, Benzene, Methane, Ammonia, Methanol, Acetone, Acetic Acid, Toluene, Phenol, Pyridine, THF, and Hexane. There are also links for 'PubChem' and 'Novoprolabs 3D', and a 'Periodic Table' button.
- Molecule Preview:** Shows a 3D ball-and-stick model of a molecule. Below the model are buttons for 'Add', 'XYZ', 'SMILES', 'Clear (3)', and a menu icon.

The bottom section is titled 'Sampling Mode' and includes a 'MAXPRINT' toggle. It contains three panels: 'Preliminary' (Stop after geometry optimization), 'Rigorous' (Stop after geometry refinement), and 'Ultimate' (Stop after energy refinement). At the very bottom, a horizontal flowchart shows the workflow: 'Annealing' → 'Geometry Optimization' → 'COSMIC 1' → 'Geometry Refinement' → 'COSMIC 2' → 'Energy Refinement' → 'COSMIC 3' → 'Results'.

Figure 1: Web Input Generator interface with 3D visualization and parameter forms.

3.4 Before You Download

Check these points in the generator before running:

- **Charge and multiplicity.** The generator proposes a multiplicity, which can be wrong for ions and open shell systems.
- **Box.** Confirm it with the Visual Test: the molecules should neither be compressed nor lost in empty space.
- **Conformational sampling.** Enable it when the molecules have flexible dihedrals.
- **Resources.** Concurrent jobs times cores per job must not exceed your machine, and the memory in the ORCA templates must fit it.
- **Mode.** Start with a preliminary run on a new system, and do a review run before an expensive rigorous or ultimate one.
- **Pipeline summary.** It should list exactly the stages you expect.

4 Quick Start: Glycolaldehyde and Water

This example runs a preliminary search on glycolaldehyde with two water molecules, a system with both **configurational** (arrangement of the molecules) and **conformational** (internal dihedral rotation) freedom. It needs no knowledge of the theory or the input syntax, verifies the installation, and gives a first overview of the workflow and its results in under a minute.

We begin by generating the initial coordinates with the web input generator. Glycolaldehyde is the simplest member of the diose family, so it can be located in the generator by searching for the name *diase* (Figure 2).

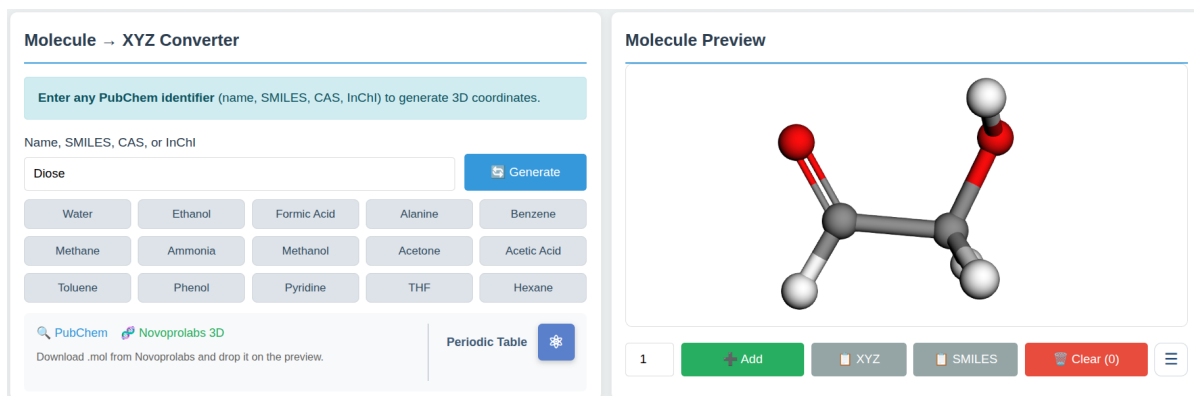


Figure 2: Generating initial coordinates for glycolaldehyde and water using the web interface.

A crucial step in the setup is superposition: both the glycolaldehyde and the water molecules are placed at the exact center of the simulation box, ensuring that the stochastic search starts from an unbiased configuration. The resulting setup should resemble Figure 3, so that the center of mass for each molecule occupies the same region of space before the simulation begins.

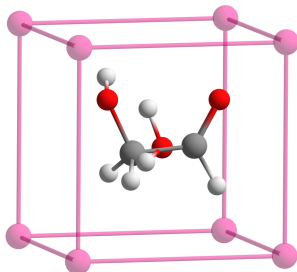


Figure 3: Initial simulation box containing glycolaldehyde and water superposed at the center.

In this example we select a preliminary run (Figure 4), which uses the GFN2-xTB method for both the single annealing stage and the subsequent candidate preoptimization; motifs are then identified by the COSMIC clustering algorithm. This is an **optimization only** workflow (no frequency calculations), which keeps the process fast but does not guarantee that the resulting structures are true local minima.

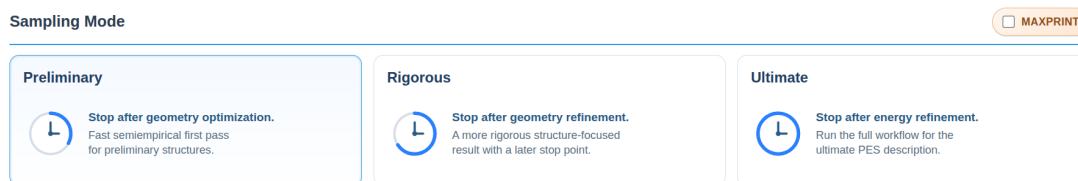


Figure 4: Preliminary sampling mode selection.

We enable the conformational option at the annealing theory level (Figure 5) and then download the `.asc` input file.

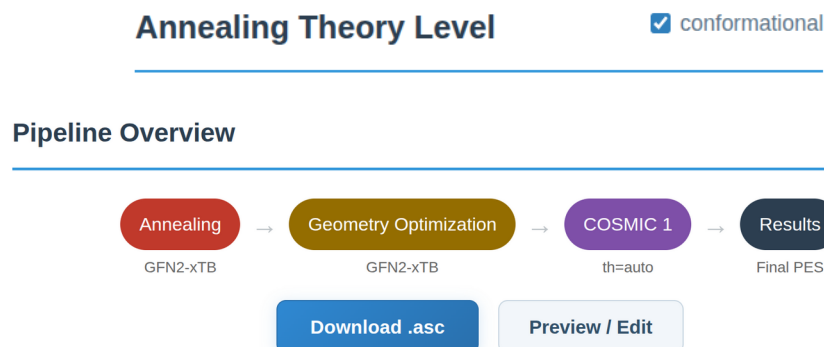


Figure 5: Conformational check and input download.

Its input, like those of the other systems in this manual, ships in the `examples/` folder of the installation. Copy it to a working folder and run it there, using eight processor cores:

```
cp -r ~/software/ascec04/examples/glyw2/preliminary glyw2 && cd glyw2 && ascec glyw2.asc
```

Upon execution, a progress bar is displayed.

```
=== COSMIC ASCEC ===
-----
Progress [.....] 0.0%
-----
[1/3] Annealing ...
```

Here, [1/3] indicates that the program is in the first of three main stages: Annealing, Geometry Optimization, and COSMIC screening.

Once the run is complete, the terminal will display a summary message.

```
=== COSMIC ASCEC ===
-----
Progress [#####] 100.0%
-----
[1/3] Annealing ✓
[2/3] geometry_optimization ✓
[3/3] cosmic (80→37) ✓

Workflow finished
Start: 2026-06-23 17:43:40
End:   2026-06-23 17:44:17
Total wall time: 36s 835ms
-----
Warning: No true minima can be assured without frequency calculations.
Protocol summary saved to protocol_summary.txt

✓ Miniprint: 23.8 MB (use --maxprint to keep all files)
```

In approximately 37 seconds, ASCEC autonomously executed a three stage pipeline:

1. **Annealing** (66.3%): The stochastic search explored the configurational and conformational space of glycolaldehyde with two waters using GFN2-xTB energies. Starting from a “Big Bang” superposition, it generated **80 accepted configurations** while cooling the system from 300 K down to near 0 K.
2. **Geometry Optimization** (33.7%): All 80 candidate structures were optimized at the GFN2-xTB level, relaxing each configuration into the nearest local energy basin and producing a set of preoptimized structures.
3. **COSMIC** (< 0.1%): The clustering algorithm analyzed the 80 optimized structures using a reduced physicochemical feature vector and identified **37 motifs**, the distinct families of hydrated glycolaldehyde.

A representative subset of these motifs is shown in Figure 6; each of them corresponds to a different interaction pattern between water and glycolaldehyde, with a certain degree of redundancy remaining.

The warning in the terminal output appears because a preliminary run computes no frequencies, so its structures are candidates rather than confirmed minima. For verified minima and Boltzmann populations, use the rigorous input (`examples/glyw2/rigorous`), as in the water hexamer example that follows (Section 1.1).

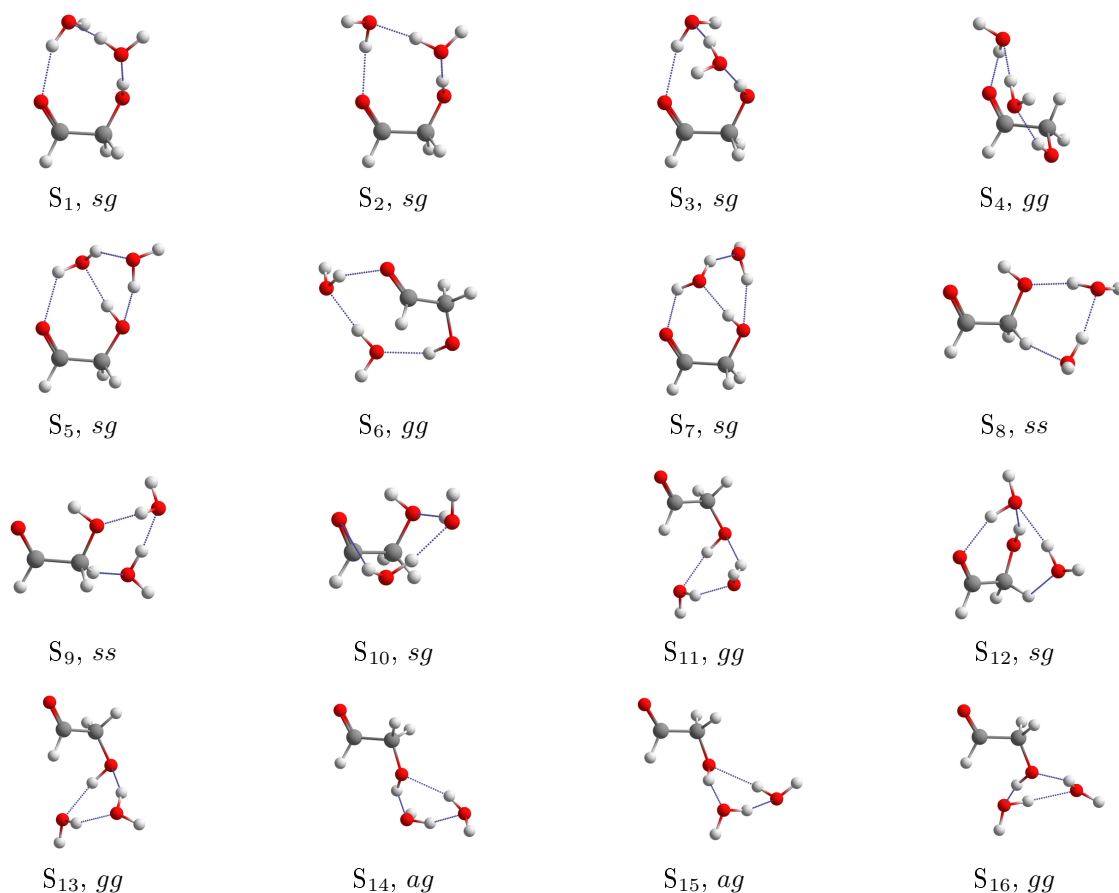


Figure 6: First 16 of the 37 preliminary glycolaldehyde water motifs. Labels give the glycolaldehyde conformation, first the C-C and then the C-O dihedral: *s* *syn*, *g* *gauche*, *a* *anti*.

5 Water Hexamer

This example builds a rigorous run of the water hexamer (H_2O)₆, a well studied benchmark [5], from scratch in the web generator. Unlike the Quick Start, it adds a geometry refinement with frequencies, which gives verified minima and Boltzmann populations.

5.1 Molecular Coordinates Generation

Open the Web Input Generator at <https://manuel2gl.github.io/qft-cosmic-ascec/> (or run `ascec input` in the terminal). Using Smart Mode, search for "water" in PubChem to fetch its 3D structure, then add six copies to the simulation box (Figure 7).

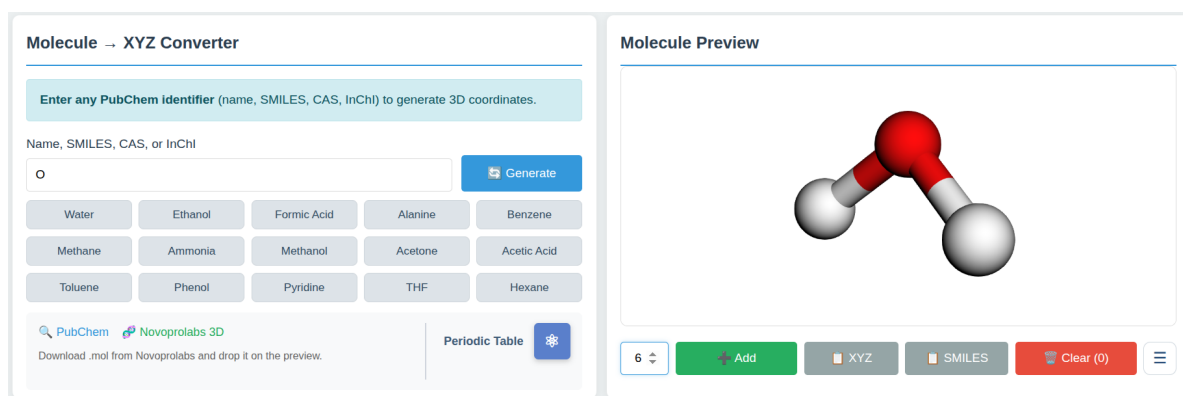


Figure 7: Generating six water molecules using the PubChem integration.

5.2 Mode Selection

Select the **rigorous** mode (Figure 8; Section 1.1).

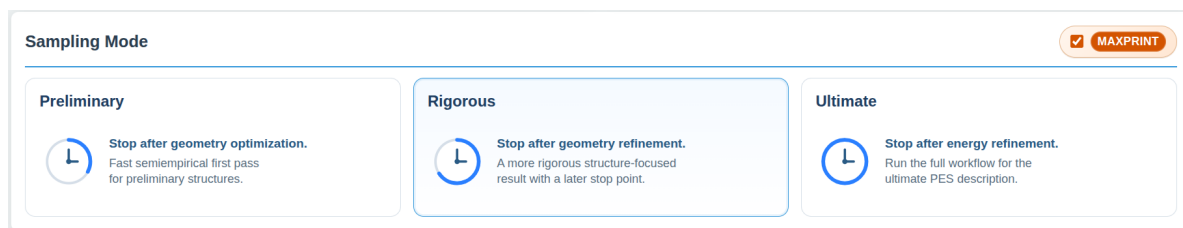


Figure 8: Rigorous mode selection.

As shown in Figure 8, the **maxprint** option is enabled by default in rigorous and ultimate modes: because these workflows involve expensive calculations, the user may prefer to retain all intermediate I/O files for further analysis. If it is disabled, **miniprint** is used instead and only the I/O files of the final ensemble are kept, saving disk space.

5.3 Simulation Box

Rigorous runs use a packing of $\phi = 20\%$ by default (30% for preliminary ones), which leaves the molecules room to reorganize. For six waters the generator suggests a box of about 7.3 Å (Figure 9); how the length is computed is described in Section 8.

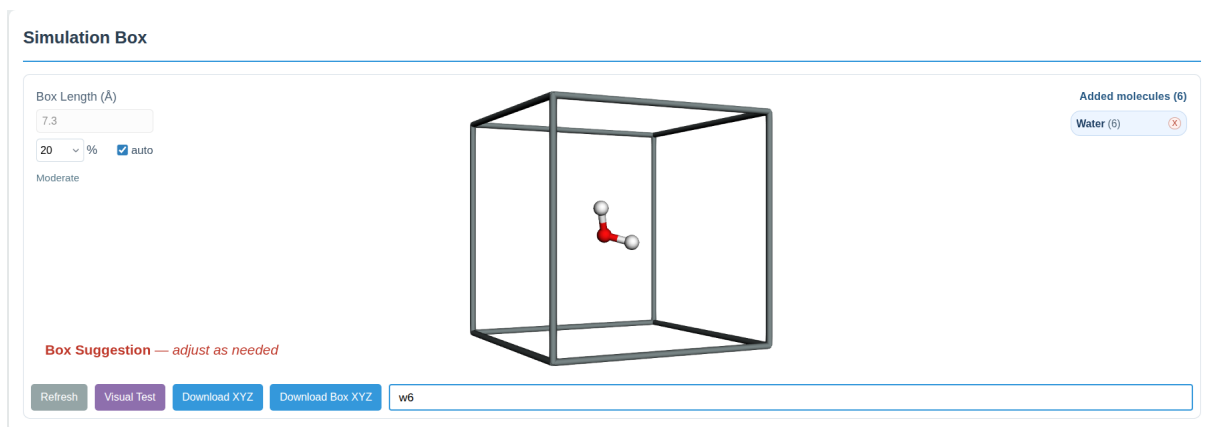


Figure 9: Simulation box setup for the water hexamer at 20% effective packing ($L \approx 7.3$ Å).

Check the box with the Visual Test (Figure 10), which mimics the random rigid body moves of the annealing: a compressed box leaves no room for the cluster to form, and an oversized one leaves the molecules unable to meet.

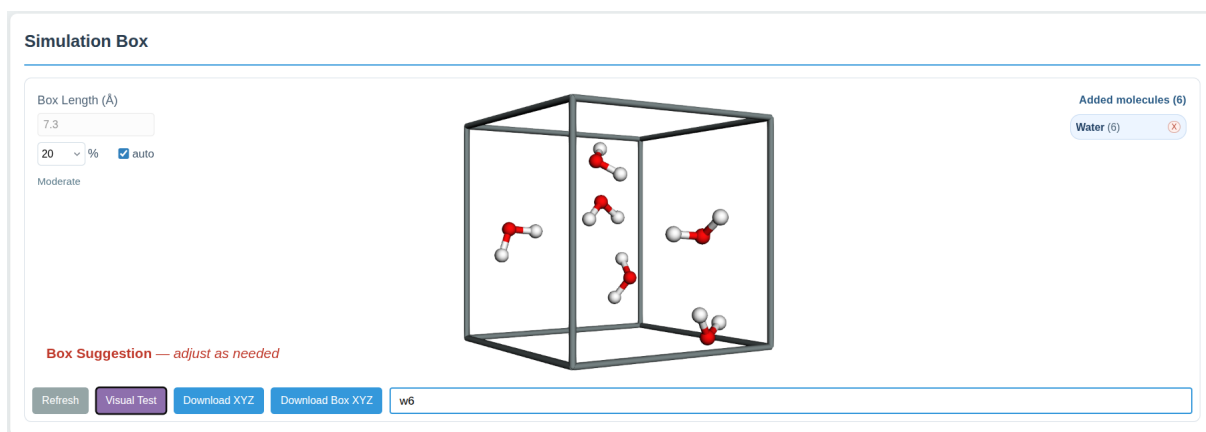


Figure 10: Water hexamer simulation box visual test

5.4 Annealing Parameters

Smart Mode provides a guided interface for selecting the annealing settings (Figure 11).

It supplies sensible defaults that the user can override as needed. The key parameters are:

- **Conformational:** Controls sampling of internal (torsional) degrees of freedom. For a *single* fragment it is enabled automatically with a sampling percentage of 100%. For *multiple* fragments it is optional (off by default); when enabled it defaults to 30%.
- **Annealing method:** GFN2-xTB semiempirical, for fast energy evaluations during the stochastic search.
- **nprocs:** Number of processors used for the QM calculations.
- **Concurrent & Replicas:** three replicas run at the same time, for better coverage of the surface.
- **Charge & Multiplicity:** The system's overall charge and spin multiplicity; the GUI proposes a multiplicity, but it should always be reviewed.

- **Temperature schedule:** Geometric quenching, which gradually lowers the acceptance probability of high energy states.
- **Box:** With auto box enabled, the packing fraction can be adjusted; with it disabled, a custom box length can be entered.

Annealing Theory Level				Quenching & Box Settings			
☐ conformational				☒ auto box			
Theory	Method	Basis		Route	Initial T (K)	Factor (%)	Steps
Standalone ...	GFN2-xTB	(none)		Geom...	500	5	100
nprocs	Concurrent	Charge	Mult.	Replicas	Packing (%)	Box (Å)	
1	3	1	1	3	20	10.0	

Figure 11: Annealing parameter selection.

5.5 Geometry Optimization and Refinement

A geometry optimization is first carried out with the GFN2-xTB semiempirical method to relax the accepted structures and enable a first COSMIC screening.

The refinement step reoptimizes the unique motifs identified by the first clustering pass at a higher level of theory (Figure 12). For this run we use **wB97X-D3BJ/def2-TZVPP** with a full **Opt+Freq** treatment, which ensures that the final structures and their relative energies are accurate enough for meaningful thermodynamic analysis. Crucially, the frequency calculation confirms each structure as a true local minimum.

A second COSMIC analysis is then applied to the refined ensemble to produce the final set of unique motifs and the Boltzmann population distribution.

Geometry Optimization				Geometry Refinement			
				☒ optional			
Theory	Method	Basis		Theory	Method	Basis	
Standalone ...	GFN2-xTB	(none)		DFT	wB97X-D3BJ	def2-TZVPP	
nprocs	Concurrent	MaxIter	Job	nprocs	Concurrent	MaxIter	Job
1	8	200	Opt	12	2	120	Opt Freq

Figure 12: Geometry optimization & Refinement theory level.

5.6 Energy Refinement and COSMIC Screening

An optional energy refinement is reserved for the ultimate mode, which is why it is disabled in this rigorous run. The COSMIC screening parameters are also set here: in **automatic** mode the protocol determines the best clustering threshold during the first COSMIC pass (after the geometry optimization stage; see Section 6.3.7 for details) and reuses that same threshold for the final COSMIC stage (Figure 13).

Energy Refinement ☐ optional			COSMIC Screening ☐ same for opt/ref		
Theory	Method	Basis	Opt th	Opt RMSD	Opt -j
Post-HF	DLPNO-CC...	aug-cc-pVTZ	automatic	off	4
nprocs	MaxCore (MB)	Job	Ref th	Ref RMSD	Ref -j
1	50000	SP (Single ...)	opt	off	4

Figure 13: Energy Refinement & COSMIC Screening.

5.7 Pipeline Review

Before download, the pipeline summary lists every stage of the protocol (Figure 14).

Pipeline Overview



Figure 14: Complete pipeline review showing all workflow stages.

5.8 Running and Monitoring the Job

Download the .asc file to your working folder. Before running, adjust the ORCA templates at the bottom of the file to your machine, mainly `nprocs` and, when needed, `%maxcore` for the memory:

```

#orca input2
#name
#rescue(HF-3c/freq)
! wB97X-D3BJ def2-TZVPP def2/J RIJCOSX DEFGRID3
! Opt Freq TightOpt TightSCF
%pal
  nprocs 12
end
%geom
  maxiter 120
end
* xyz 0 1
#
*

```

Then run the protocol:

```
ascec w6.asc
```

A full run takes hours to days, so it is normally left running unattended. Launch it in the background with `ascec w6.asc &`, or press `Ctrl+D` (or `Ctrl+Z`) while it runs: the program relaunched itself detached from the terminal and prints its PID. Closing the terminal does not stop a detached job.

The `ascec status` command opens an interactive viewer of all ASCEC jobs on the machine (a registry shared across terminals), so a detached run can be inspected at any time:

```
ascec status
```

It lists the running and recent jobs with their ID, PID, input file, and start time. From the prompt you can attach to a job and watch its live progress (`V<id>`), terminate it (`K<id>`), refresh the list (`R`), or quit the viewer (`Q`), which leaves the jobs running.

```
ASCEC STATUS (2026-06-25 20:28:46)

Running:
ID      PID      INPUT FILE      STARTED
---      -
1      1131777  w6.asc          2026-06-24 16:54:48

-----
[V <id>] Attach/View  [K <id>] Kill      [R] Refresh      [Q] Quit
-----
Choice: v1
```

```
=== COSMIC ASCEC ===
-----
Progress [#####.....] 68.6%
-----
[1/5] annealing ✓
[2/5] geometry_optimization ✓
[3/5] cosmic ✓ (215→39)
[4/5] geometry_refinement 21/39 ...

Attached: job 1 (w6.asc)   updated: 2026-06-25 18:29:05
Ctrl+C or Ctrl+D to detach (job keeps running)
Input realpath: /home/user/w6/w6.asc
```

The run is resumable. Each stage checkpoints its progress to a cache file, so if the workflow is interrupted (or deliberately stopped), simply reissuing the same command, `ascec w6.asc`, picks up where it left off: completed stages are read from the cache and skipped, and execution continues from the first unfinished stage.

If you have another protocol to run, you can queue it behind a job that is already running instead of waiting for it to finish. The **after** keyword takes the PID of the running job (shown in **ascec status**) and holds the new run until that job completes.

```
ascec input2.asc after 1131777
```

```
ASCEC queued in background (PID 3964514) - holding until PID 1131777 finishes,
then it starts automatically.
Track or cancel: ascec status
```

5.9 Results

The three replicas accepted **215 configurations**. After the GFN2-xTB optimization, the first COSMIC pass kept **39 motifs**, which were refined at wB97X-D3BJ/def2-TZVPP with frequencies, and the final pass left **22 unique minima** (Figure 15). The first pass detected $\tau = 2.0$ (Section 6.3.7), which the final pass reused. The redo loop cleared the 23.1% of refined structures first flagged as critical, so all 22 are true minima. The five lowest motifs, within 0.6 kcal/mol of the global minimum, hold about 88% of the Boltzmann population at 298.15 K (Figure 16). The run took about 1 day and 3 hours on a workstation.

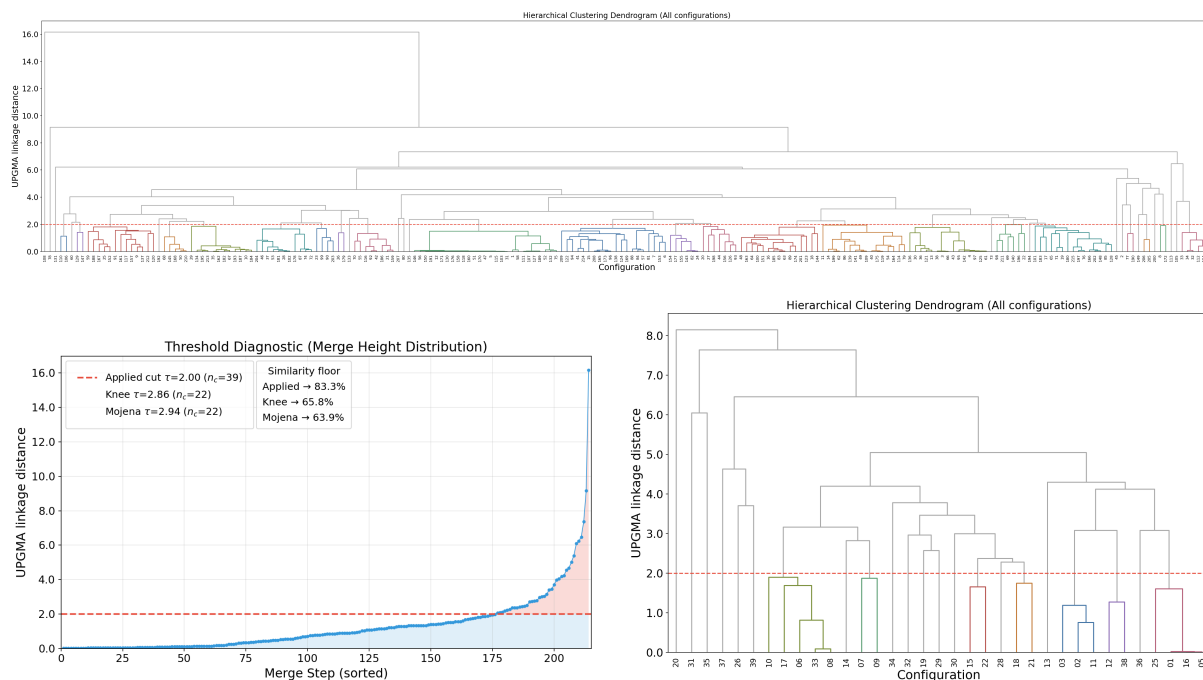


Figure 15: COSMIC analysis of the water hexamer. Top: dendrogram of the first pass, 215 preoptimized structures grouped into 39 motifs. Bottom: the threshold diagnostic of that pass, which selected τ , and the dendrogram of the final pass, 39 refined structures grouped into 22 unique motifs.

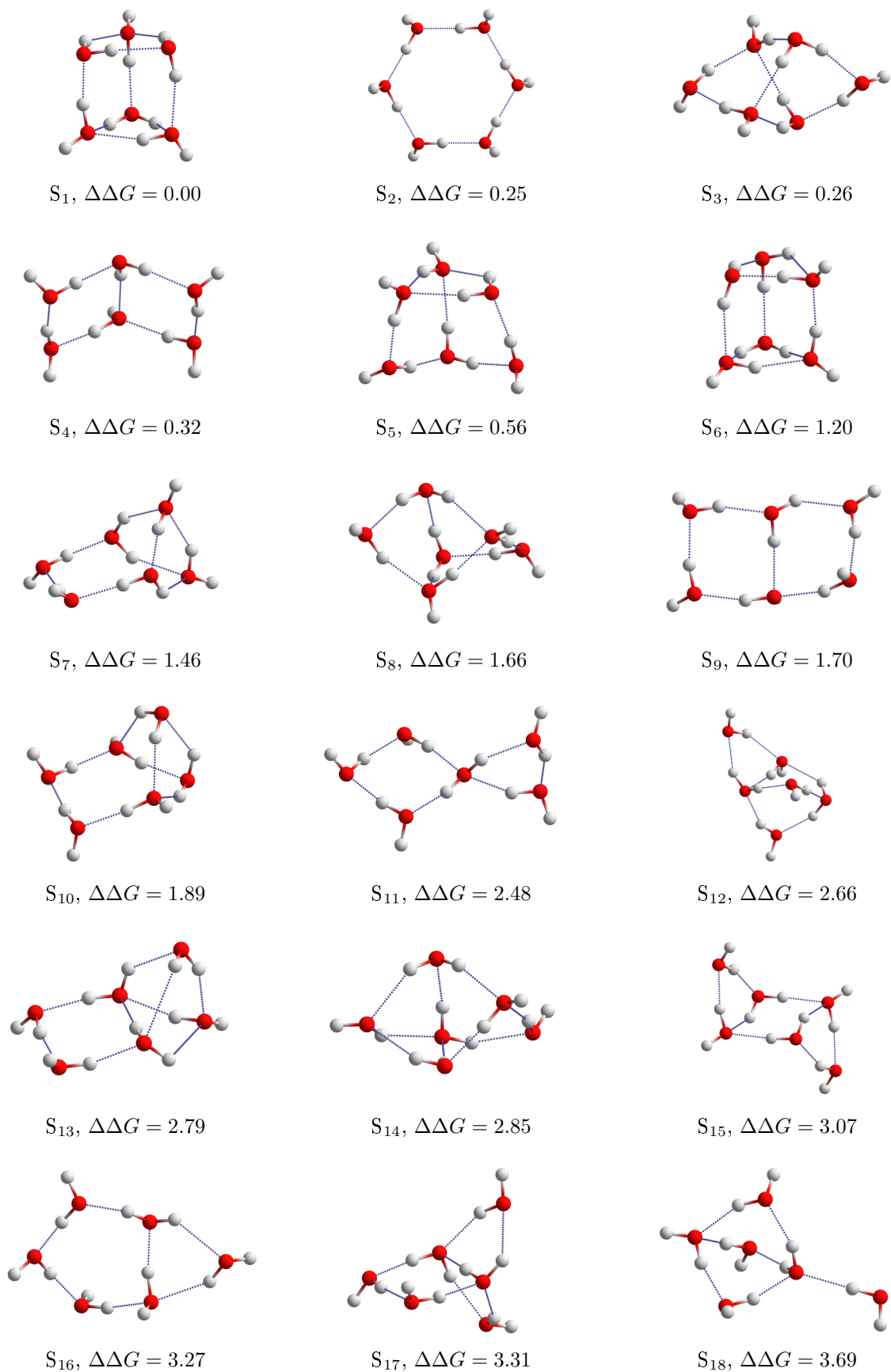


Figure 16: First 18 of the 22 refined water hexamer minima, labeled with their relative Gibbs free energy $\Delta\Delta G$ (kcal/mol) at 298.15 K.

Where the time goes. Figure 17 gives the wall time of every benchmark run, all on the same 24 core Intel Xeon workstation. Absolute times depend on the hardware and the settings, but the proportions carry over. Preliminary runs spend most of their time on the semiempirical energies of the annealing, and none took more than about three minutes. In rigorous runs over 99% of the time goes to the DFT geometry refinement, and in ultimate runs the DLPNO-CCSD(T) energies add most of the rest. The refinement stays affordable because it starts from preoptimized geometries and only sees the structures COSMIC keeps; for the same reason extra annealing runs are cheap, and three to five replicas are recommended.

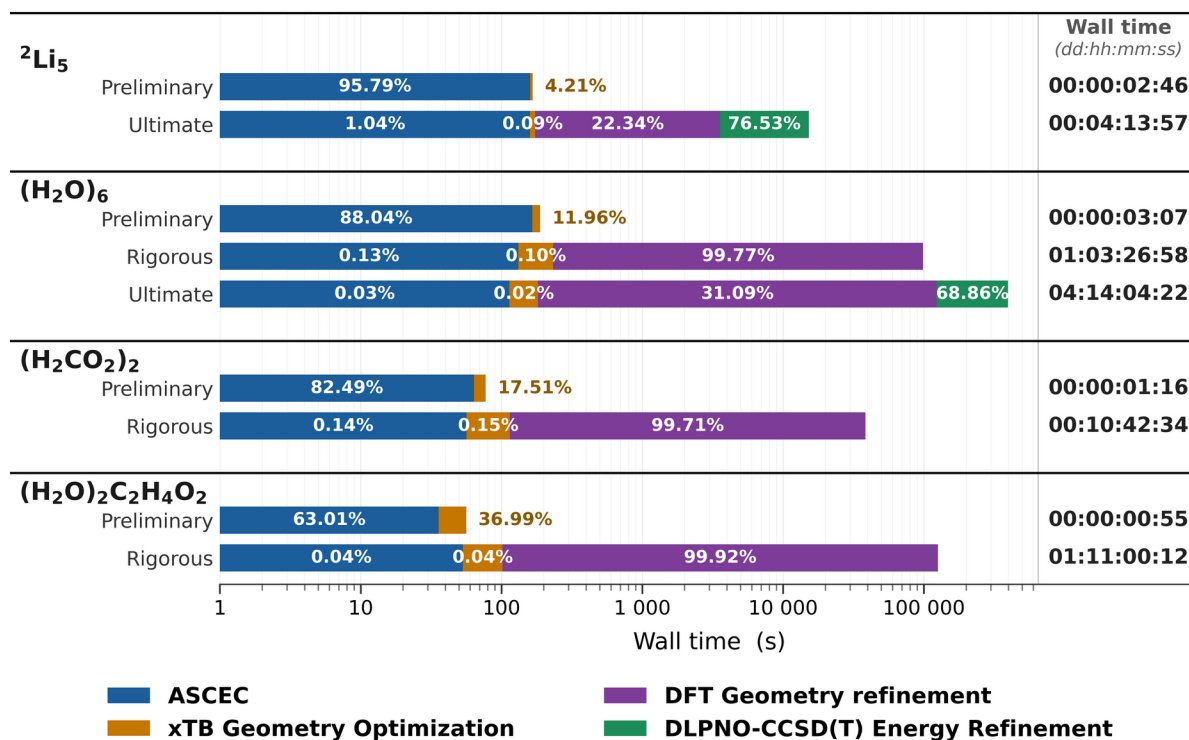


Figure 17: Wall time of each stage for the benchmark runs, on a logarithmic axis. COSMIC times are negligible and are included in each stage.

Beyond Part I. The two examples above are enough to take a new system from its first input file to a set of verified minima. Part II explains what happens along the way. It starts with the theory, the simulated annealing and the physicochemical vector that COSMIC uses to group structures, describes the input file, the protocol and outputs each run produces. It also shows how to rehearse, resume or repair a run, how to run each stage on its own, and how to use COSMIC on structures from other programs. With this understanding, the protocol can be adapted to new systems with confidence, instead of relying only on its defaults.

Part II

Advanced Usage and Reference

6 Theoretical Background

6.1 Simulated Annealing Algorithm

The ASCEC module drives the exploration of the configurational space through a Markov Chain Monte Carlo (MCMC) protocol implemented within a simulated annealing framework [6]. To ensure statistical significance, the algorithm generates thousands of trial configurations. The quantum energy of every generated structure is calculated using efficient methods (supporting any theoretical level available in the external package, e.g., GFN2-xTB, PM3, AM1, HF).

The acceptance of a structural perturbation depends on the change in potential energy, $\Delta E = E_{new} - E_{old}$. If the structural change lowers the energy ($\Delta E < 0$), the move is accepted immediately. In the case of an energy increase ($\Delta E > 0$), the acceptance behavior depends on the chosen criterion:

Standard Metropolis Criterion [7] (Optional) When explicitly requested (via `--standard` flag), the algorithm follows the classical Boltzmann probability. An uphill move is accepted if a random number $\xi \in [0, 1]$ satisfies:

$$\xi < e^{-\frac{\Delta E}{k_B T}} \quad (1)$$

Modified Metropolis Criterion (Default) To mitigate kinetic trapping in local basins and prevent jumps to physically unrealistic high energy configurations ASCEC implements a modified acceptance condition [8]. Unlike the standard probabilistic approach, which compares the Boltzmann factor to a random number, this method evaluates the relative energy cost of the perturbation. The new structure is accepted if:

$$\Phi(\Delta E) < P(\Delta E) \quad (2)$$

where $P(\Delta E) = e^{-\Delta E/k_B T}$ is the temperature dependent Boltzmann probability, and $\Phi(\Delta E)$ represents the relative energy change defined as:

$$\Phi(\Delta E) = \frac{\Delta E}{|E_{new}|} \quad (3)$$

This criterion was found to be more adequate for this class of problems. By removing the stochastic element found in the standard Metropolis method, the algorithm avoids the accidental rejection of chemically viable structures due to random number generation, while still filtering out moves where the relative energy increase is drastic.

Temperature Schedules: The simulation temperature decreases over n_T steps to gradually reduce the acceptance of high energy states. A choice between a linear (decreasing T by a constant amount) and a geometric (decreasing T by a constant percentage) route is available:

- *Linear Quenching:* The temperature is reduced by a fixed amount dT at each step:

$$T_{n+1} = T_n - dT$$

- *Geometric Quenching:* The temperature is scaled down by a cooling factor f (in %):

$$T_{n+1} = T_n \times \left(1 - \frac{f}{100}\right)$$

6.2 Monte Carlo Moves

The configuration space is explored using three perturbation motions:

1. **Translation:** Random displacement of the center of mass.
2. **Rotation:** Random rotation around principal axes.
3. **Conformational (Optional):** Modification of internal dihedral angles.

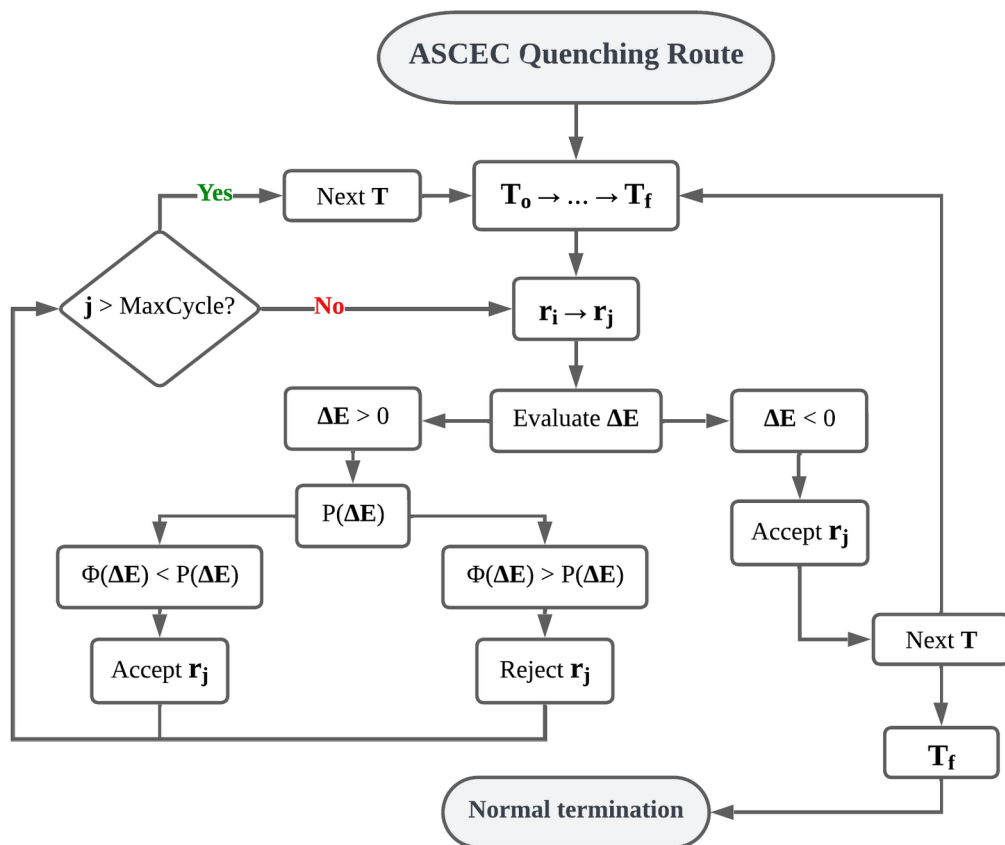


Figure 18: SA procedure using a modified Metropolis criterion.

6.3 Topological Clustering (COSMIC)

While stochastic exploration (ASCEC) is effective at traversing energy barriers, it inherently suffers from a data management limitation: the generation of highly redundant datasets. A typical annealing run yields hundreds of candidate structures, the vast majority of which converge to identical basins of attraction. To address this, the **COSMIC** module functions as a postprocessing discriminator designed to filter, validate, and cluster the raw output into unique chemical motifs.

The scope of this algorithm is to bridge the gap between stochastic chaos and ordered chemical insight. By reducing massive datasets into nonredundant families, it facilitates the manual revision of the Potential Energy Surface (PES) and ensures that only topologically distinct minima are subjected to high level quantum mechanical refinement.

6.3.1 The Physicochemical Feature Vector

Standard clustering methods often rely solely on geometric superposition (RMSD), which depends on atomic indexing [9]. To overcome this, we define the configurational identity of a structure i using a multidimensional scalar feature vector, $\mathbf{x}_i$. This vector aggregates continuous physicochemical descriptors derived from the quantum mechanical data. The **full vector** (up to 15 dimensions) is:

$$\mathbf{x}_i = (E_{elec}, V_{nuc}, G, E_{HOMO}, \Delta E_{HL}, \mu, A, B, C, \nu_{smallest}, \nu_{largest}, n_{HB}, \overline{d_{HB}}, \overline{\theta_{HB}}, \sigma_{d_{HB}}) \quad (4)$$

where the components represent energetics (E_{elec} , G); V_{nuc} : nuclear repulsion energy; E_{HOMO} : energy of the HOMO; ΔE_{HL} : HOMO–LUMO gap; μ : dipole moment; A, B, C : rotational constants; $\nu_{smallest}, \nu_{largest}$: smallest and largest vibrational frequencies; n_{HB} : number of hydrogen bonds, and hydrogen bonding descriptors $\overline{d_{HB}}, \overline{\theta_{HB}}, \sigma_{d_{HB}}$: geometrical parameters defining hydrogen bonds within each structure.

All structures are clustered together in a single pool by default; they are not grouped by hydrogen bond count beforehand. H-bond information is encoded indirectly through $\overline{d_{HB}}$ and $\overline{\theta_{HB}}$, allowing the clustering algorithm to account for H-bond patterns without imposing rigid partitions. The optional `--group-hb` flag restores per H-bond count grouping.

6.3.2 Weighted COSMIC Metric

Since the vector components possess disparate physical units, a per feature Z score standardization is applied $z_i^k = (\gamma_i^k - \bar{\gamma}^k)/\sigma^k$, $Z_i^k = w^k \times z_i^k$. The distance between two structures is then quantified using the Euclidean distance:

$$d(\mathbf{X}'_i, \mathbf{X}'_j) = \sqrt{(\mathbf{X}'_i - \mathbf{X}'_j) \cdot (\mathbf{X}'_i - \mathbf{X}'_j)} = \sqrt{\sum_{k=1}^{n_d} (Z_i^k - Z_j^k)^2} \quad (5)$$

where w^k represents user definable weights used to bias the clustering focus (e.g., prioritizing H-bonding patterns over soft vibrational modes).

6.3.3 Hierarchical Two Step Clustering

To ensure both computational efficiency and high structural fidelity, the clustering process operates via a hierarchical "funnel" protocol involving two distinct steps:

1. Step I: Physicochemical Clustering (Coarse Grained).

The algorithm first performs agglomerative hierarchical clustering using UPGMA (average linkage) based on the standardized z score feature vectors $\mathbf{x}$. This rapidly groups configurations that share similar energetics, electronic states, and general shapes into broad families. The dendrogram is cut at a user defined distance threshold τ to obtain a flat partition. This step drastically reduces the dataset size by merging redundant snapshots without performing expensive geometric alignments.

2. Step II: Optional RMSD Refinement (Fine Grained).

Optionally, a secondary geometric check is applied within each identified cluster. A Root Mean Square Deviation (RMSD) analysis [10] is performed to distinguish stereoisomers or isoenergetic conformers that map to identical scalar vectors (e.g., enantiomers or distinct proton ordering networks with identical energies). This ensures that subtle geometric variations are not lost during the vector based reduction.

6.3.4 Feature Vector Technicalities

Three implementation details govern how the raw physicochemical descriptors of Eq. 4 become the standardized vectors that are actually clustered.

Hydrogen bond descriptors. The components n_{HB} , $\overline{d_{\text{HB}}}$, $\overline{\theta_{\text{HB}}}$ and $\sigma_{d_{\text{HB}}}$ are computed from a strictly geometric hydrogen bond test applied to each structure. For every H covalently bound to a donor D (N, O or F), an acceptor A (also N, O or F) is counted as hydrogen bonded when the $\text{H}\cdots\text{A}$ distance lies in $[1.4, 3.2]$ Å and the $\text{D}-\text{H}\cdots\text{A}$ angle is at least 120° . These are the same criteria used throughout the cluster literature. Note that this detection is deliberately *stricter* than the broad, count based donor/acceptor heuristic used only to *size the box* (Section 8): the box heuristic overcounts on purpose to leave room, whereas the feature vector records the H-bonds a structure actually forms.

Small standard deviations. Z-standardization divides each descriptor by its standard deviation across the pool. If a descriptor is essentially constant, that division would inflate tiny numerical noise into enormous z -scores and dominate the Euclidean distance. COSMIC guards against this by *dropping* any feature column whose population standard deviation falls below a minimum threshold, or whose total spread across the pool is smaller than a user set absolute tolerance. A dead column carries no discriminating information, so removing it entirely (rather than retaining it as noise) keeps the distances finite and meaningful; the dropped features are listed in the verbose log.

Pearson similarity floor. For two weighted, standardized z score vectors the Euclidean distance and the Pearson correlation r between their profiles are linked by $d^2 = 2 N_f (1 - r)$, where N_f is the effective number of active features (the sum of the column variances after weighting). COSMIC inverts this, $r = 1 - d^2/(2N_f)$, to report, for every cluster, the lowest percentage similarity between any member and its representative — the **similarity floor**. A high floor (e.g. $> 90\%$) certifies that a family is internally homogeneous, giving a direct, quantitative check on the clustering quality.

Partial weights (preliminary runs). By default every feature carries equal weight ($w^k = 1$), which keeps COSMIC method agnostic. The flag `--partialweights` instead applies a tuned, reliability based profile that downweights the descriptors most sensitive to method noise and atom indexing — dipole moment (0.6), the hydrogen bond descriptors (0.7), the HOMO energy and highest vibrational frequency (0.85) — while leaving the deterministic geometric quantities (electronic energy, nuclear repulsion, rotational constants) at 1.0, so that geometrically identical structures do not look artificially distinct at a noisy level of theory. The web input generator adds `--partialweights` automatically to the COSMIC step of *preliminary* runs (where the whole pipeline stays at the semiempirical/xTB level); it is a deliberate choice for that specific case, not a blanket setting for every semiempirical calculation. An explicit `--weights` argument always layers on top.

6.3.5 A Dynamic Feature Vector

The feature vector is dynamic, not a fixed template. COSMIC reads whatever properties each quantum chemistry output actually contains and clusters on the **largest feature pool the dataset supports**: electronic and orbital energies, the HOMO–LUMO gap, dipole, nuclear repulsion, rotational constants, hydrogen bond descriptors, and, when the calculation included frequencies, the Gibbs energy and vibrational frequencies. Any descriptor a structure lacks, or that carries no variance across the pool (the small standard deviation rule above), is left out automatically. The motifs are therefore always identified from the richest description available for that dataset, with no need for the user to declare in advance which properties exist.

6.3.6 Quality Control

Beyond clustering, the module serves as a topological filter. It screens for the presence of **one or more imaginary frequencies**, indiscriminately identifying first order saddle points (transition states) and higher order instabilities. If such a structure clusters tightly with a true minimum, it is discarded as a redundant, nonconverged distortion. However, if it forms an isolated cluster, it is flagged as a unique topological feature and output for reoptimization, ensuring that potential basins are not overlooked merely due to incomplete convergence.

6.3.7 Automatic Threshold Selection

By default COSMIC does **not** ask you to pick τ . It is detected per case from the merge heights of the UPGMA [11] tree, because no single fixed value suits every system.

Heights are full distances: no factor of $\frac{1}{2}$. COSMIC builds the tree with SciPy's average linkage routine and cuts it on the *distance* criterion, so the dendrogram height axis and every threshold, τ , the merge heights, and the empirical 2.0, are the full average intercluster distance of Eq. 5. Textbook UPGMA usually plots a node at *half* that value, the branch length from the node down to its leaves; SciPy, and therefore COSMIC, does not. A height read off a COSMIC dendrogram is the distance itself, directly comparable with τ .

Three criteria are available:

- **Knee** [12] (*the default*). Sort the merge heights and plot them against merge number. The curve stays nearly flat while structures belonging to one motif fuse, then climbs steeply once distinct motifs begin to merge; the elbow marks that transition and is the natural place to cut. COSMIC rescales both axes to $[0, 1]$, joins the first and last merges with a secant, and takes the merge lying furthest from it. The cut goes midway between that merge and the next, so the elbow merge itself still counts as within cluster. The knee is trusted only when the tree can support it — at least eight merges, a height range spanning a factor of five or more, and a resulting cluster count between two and $n/2$ — and otherwise COSMIC falls back to the empirical value.
- **Empirical**, $\tau = 2.0$. The fallback when a set is too small for the knee to be reliable, and also an **upper limit**: a detected knee above 2.0 is capped there. *There is no formal justification for this number.* It was determined empirically over many unrelated systems, and across 50 independent preliminary runs on the water hexamer the average knee came to 1.93, close enough that either value delivered much the same surface. Its scale is not arbitrary, though: the distance per component is $\tau/\sqrt{N_f}$, which for a full 15 component vector sits just above 0.5 standard deviations and for the sparsest 5 component one just below 1.0.
- **Mojena** [13]. Computed and drawn on the validation plots as a diagnostic, but not part of the fallback chain: across many test cases it proved too inconsistent to drive the selection.

The threshold actually applied, and where it came from (**knee**, **empirical** or **user**), is reported in the verbose log and on the diagnostic plot. To take the choice back, pass a number to `--threshold`, which is used as given and never capped, or `--th=knee` to keep the automatic detection with the 2.0 ceiling switched off.

7 Input File Format

The ASCEC input file (typically saved with an `.asc` extension) is a comprehensive "master" document. To make it easy to read and edit, it is logically divided into three main blocks: the **Simulation Parameters**, the **Execution Protocol**, and the **Embedded QM Templates**.

Note: The easiest way to generate this file without worrying about formatting is by using the [ASCEC Web Input Generator](#).

7.1 Advanced Generation

The Advanced Generation page of the web input generator (Section 3) exposes every field of the input file on one page, grouped the way the file itself is laid out. Use it when the Smart Mode defaults are not what you want, or when you need a setting Smart Mode does not offer. Nothing is hidden and nothing is inferred: what you see is what gets written. The page is organized into the following blocks.

Annealing Theory Level Theory, method and basis set, the QM program and the command used to call it, processor counts for the QM job and for ASCEC itself, how many jobs run concurrently, and the system's charge and multiplicity.

Conformational Search Whether internal degrees of freedom are sampled, the sampling percentage, and the maximum dihedral change.

Quenching and Packing Simulation mode, the linear or geometric route with its initial temperature, factor or step size and number of steps, the Monte Carlo cycles per temperature and their floor, maximum displacement and rotation, the packing percentage or an explicit box length, the number of replicas, and the molecule count.

The three calculation stages One block each for Geometry Optimization, Geometry Refinement and Energy Refinement, each with its own processors, memory, job type and iteration limit, and each with a **live input preview** showing the template exactly as it will be embedded in the `.asc` file. The refinement stages are optional, which is how the same page produces a preliminary, rigorous or ultimate protocol. When a stage uses standalone xTB, its block adds implicit solvent and extra xTB flag controls (Section 7.3.2).

COSMIC Screening The threshold, parallel jobs, whether RMSD refinement runs, and any extra flags, set independently for the pass after optimization and the pass after refinement, or locked together with one switch. A weights panel lets individual feature vector components be reweighted (Section 6.3).

Recovery The recovery strategy, the critical percentage that triggers it, and the redo limit (Section 10.5).

A **Pipeline Overview** at the top of the page lists the stages your current settings produce, in order, so the protocol can be checked before download. Anything the controls do not cover can be typed directly into the previews, or edited in the `.asc` file afterwards.

7.2 Block 1: Simulation Parameters and Coordinates

The first block of the input file relies on strict positional formatting. It defines the physical conditions of the annealing simulation and the initial coordinates of the molecules.

```
# Input for water pentamer Simulation
```

```

1 10          # Line 1: Simulation Mode & Number of Config
              # 1: Annealing; 0: Random configurations
7.0          # Line 2: Simulation Cube Length (Angstroms)
2           # Line 3: Annealing Quenching route
              # (1: Linear, 2: Geometrical)
100.0 10.0 50 # Line 4: Linear Quenching Parameters (Ti, dT, steps)
500.0  5.0 100 # Line 5: Geometric Quenching Parameters (Ti, %, steps)

3000 10      # Line 6: Max MC Cycles per T and floor value
1.0 1.0      # Line 7: Max Displacement (A) & Max Rotation (radians)
0 120       # Line 8: Conformational sampling (%) & Max dihedral (deg)

3 xtb       # Line 9: QM Program Index & alias
              # (1: Gaussian, 2: ORCA, 3: xTB)
GFN2-xTB    # Line 10: Hamiltonian & Basis Set
1 4         # Line 11: nprocs (QM calculations and ASCEC evaluation)
0 1         # Line 12: Charge & Spin Multiplicity

5           # Line 13: Number of Molecules (nmo)

# Lines below: Molecule Definition
# (Num Atoms, Label, then xyz coordinates, separated by *)

*
3
Water1
O   -0.294733  -0.218200  0.154167
H   -0.017333  0.674700  0.408567
H    0.312067  -0.456500  -0.562733
*
3
Water2
O   -0.294733  -0.218200  0.154167
H   -0.017333  0.674700  0.408567
H    0.312067  -0.456500  -0.562733
*

... (Water 3-5)

*

```

Table 3: Detailed description of ASCEC input parameters (Lines 1-13).

Line	Parameters	Description
1	[mode] [nconfigs]	1 : Annealing, 0 : Random Generation. nconfigs is only used if mode is 0.
2	box_length	Length of the cubic simulation box edge in Å.
3	route	Temperature schedule: 1 (Linear) or 2 (Geometric).
4	[Ti] [dT] [steps]	Linear: Start Temp (K), Step size (K), Number of steps.
5	[Ti] [fac] [steps]	Geometric: Start Temp (K), Reduction factor (%), Steps.
6	[max] [floor]	Maximum Monte Carlo cycles per temperature step and floor value as it decreases (0.1) with each step.
7	[disp] [rot]	Max translation distance (Å) and max rotation (radians). Maximum displacement for each mass center, and maximum rotation of the principal axes of each molecule. For monatomic species the rotation has no effect and the value is ignored.
8	[prob] [dih]	Probability of conformational move (%) and max dihedral change (°).
9	[code] [alias]	QM Program ID (1 =Gaussian, 2 =ORCA, 3 =xTB) and the command alias to run it.
10	[method] [basis]	Hamiltonian/Method (e.g., gfn2 , b3lyp) and basis set (e.g., def2-SVP). For semiempirical methods the basis field is ignored; consult the ORCA manual for the correct syntax.
11	[cpu_Q] [cpu_A]	Processors passed to the QM program, and (optional) processors for ASCEC logic and parsing.
12	[chg] [mult]	Total system Charge and Spin Multiplicity.
13	nmol	Number of molecules defined. For atomic clusters each atom must be considered a molecule. If atoms interacting with molecules are considered, then each atom not being part of a molecule must be considered a molecule.

7.3 Block 2: The Automated Protocol and Embedded Templates

Following the molecular coordinates, the file contains the **#Protocol** block and the **embedded QM templates**. The protocol is a sequence of comma separated steps. The example below is a complete ultimate protocol; stopping after the first **cosmic** gives a preliminary run, and after the second a rigorous one. Using dot instead of comma pauses the protocol for review and resume.

```
# Protocol
# (comma = continue, dot = pause; --flags apply to the whole run)

.asc,
--maxprint,
r3 --concurrent=3,
opt --concurrent=8 input1,
cosmic -j4,
ref --concurrent=2 input2,
cosmic --th=opt -j4,
eref input3,
cosmic --th=opt -j4
```

Workflow: Annealing → Opt → COSMIC → Ref → COSMIC → Eref → COSMIC

Each optimization or refinement step names a template (`input1`, `input2`, `input3`) that is **embedded at the bottom** of the same file, using the header and placeholder syntax of Section 7.3.1. The three tiers here are GFN2-xTB (`#xtb`), wB97X-D3BJ/def2-TZVPP with frequencies (`#orca`), and a DLPNO-CCSD(T)/aug-cc-pVTZ single point (`#orca`):

```
#xtb input1
#name
#rescue(GFN2-xTB/freq)
# GFN2-xTB Opt Tight
# nprocs 1
# maxiter 200

#orca input2
#name
#rescue(HF-3c/freq)
! wB97X-D3BJ def2-TZVPP def2/J RIJCOSX DEFGRID3
! Opt Freq TightOpt TightSCF
%pal
  nprocs 12
end
%geom
  maxiter 120
end
* xyz 0 1
#
*

#orca input3
#name
! DLPNO-CCSD(T) aug-cc-pVTZ def2/J AutoAux RIJCOSX TightPNO TightSCF
%scf
  MaxIter 250
end
%pal
  nprocs 1
end
%maxcore 50000
* xyz 0 1
#
*
```

A template is an ordinary QM input file for the target program, with a few `#` placeholders that ASCEC fills in for every structure. Each template, embedded at the bottom of the `.asc` file, begins with a header line that names the program *and* gives a label the protocol refers to:

```
#orca <label>      #xtb <label>
```

A protocol step such as `opt input1` or `ref input2` then selects the block whose label matches; the header program also fixes the file extension written for that stage (`.inp` or `.xyz`). The placeholders ASCEC recognizes are shown in Table 4

Placeholder	Meaning
#orca xtb <label>	Block header: declares the QM program alias and the label to call
#name	(Optional, ORCA and xTB) replaced by the structure's comment/-filename, so each generated input is traceable.
#	(ORCA) a bare hash, placed between * xyz <chg> <mult> and the closing *, marks where the Cartesian coordinates are inserted.
!name	(Optional, Gaussian) the Gaussian spelling of #name: a comment line naming the structure.
!	(Gaussian) a bare exclamation mark on its own line marks where the coordinates are inserted.
#rescue(method[/freq][,num])	Method used to reoptimize failed structures in the redo loop (e.g. #rescue(HF-3c/freq), #rescue(GFN2-xTB/freq)).
# <flag> [args]	(#xtb blocks only) any xTB command line option, passed through as --flag args (e.g. # alpb water). See Section 7.3.2.

Table 4: Template placeholders substituted by ASCEC.

7.3.1 Input Templates

I. ORCA template. Written in native ORCA syntax. The bare # sits between * xyz <chg> <mult> and the closing *, exactly where the coordinate block is injected:

```
#orca input1
#name
#rescue(Native-GFN2-xTB/num)
! Opt GFN2-xTB TightSCF NumFreq PAL8
%geom
  maxiter 5000
end
* xyz 0 1
#
*
```

II. xTB template. For standalone xTB the run directives are given as #-prefixed lines (method, processors, iterations); ASCEC supplies the geometry automatically, so no * xyz * block is needed:

```
#xtb input1
#name
#rescue(GFN2-xTB/freq)
# GFN2-xTB Opt Tight
# nprocs 1
# maxiter 200
```

7.3.2 Advanced: the Full xTB Flag Set

Unlike ORCA or Gaussian, xTB has no keyword line and no %-block syntax: *every* setting is a command line flag. An #xtb template block is therefore not an input file but a description of

the command ASCEC will build, and each #-prefixed line contributes to it. Two layers are read, in this order.

Named directives. The first line without a recognized keyword is the **method/job line**, and a handful of directives carry names that do not match their flag. These are translated:

Directive	Becomes	Meaning
# GFN2-xTB	--gfn 2	GFN Hamiltonian. Also GFN1-xTB, GFN0-xTB and GFN-FF (→ --gfnff). GFN2 is the default.
# ... Opt <level>	--opt <level>	Geometry optimization. The level is optional and may be Crude, Sloppy, Loose, Lax, Normal, Tight, VTight (or VeryTight) or Extreme.
# ... Freq	--hess	Vibrational analysis. NumFreq is accepted as a synonym: xTB has an analytical Hessian and uses it either way.
# nprocs N	--parallel N	Cores for this job.
# maxiter N	--cycles N	Maximum optimization cycles.
# charge N	--chrg N	Total charge (omitted when neutral).
# uhf N	--uhf N	<i>Unpaired electrons</i> (2 <i>S</i>), not the multiplicity: a triplet is # uhf 2.

Table 5: Named #xtb directives and the xTB flags they produce.

Everything else. Any other # <flag> [args] line is passed through verbatim as --flag args, so the complete xTB command line is reachable from the template without ASCEC needing to know the flag. This is how solvation, electronic temperature, accuracy, and every other xTB option are requested:

```
#xtb input1
#name
#rescue(GFN2-xTB/freq)
# GFN2-xTB Opt Tight
# nprocs 1
# maxiter 200
# uhf 1
# alpb water
# etemp 400
# acc 0.5
```

which ASCEC turns into:

```
xtb <structure>.xyz --gfn 2 --opt tight --parallel 1 --cycles 200 --uhf 1 --alpb water --etemp
400 --acc 0.5
```

Common cases are the implicit solvent models, # alpb <solvent> (ALPB, the current recommendation) and # gbsa <solvent> (GBSA), optionally with a reference state, e.g. # alpb water bar1M; the # cpcmx <solvent> extended conductor like model; and single flag switches such as # json or # lmo, which need no argument.

Four rules govern the passthrough:

- **Reserved names are never passed through.** name, rescue, nprocs, maxiter, charge/chrg, uhf, mult and freq are consumed by Table 5; namespace is reserved because ASCEC in-

jects it to keep each structure’s scratch files apart, and overriding it would break output collection.

- **The protocol wins on resources.** `--parallel` and `--cycles` are reissued from the workflow settings (index line 11 and the stage `--concurrent` bookkeeping), so `# nprocs` in the template is a default, not a guarantee. Charge and multiplicity likewise come from the annealing parameters, which are the single source of truth for the system.
- **Lines carrying shell metacharacters are ignored.** The flags end up inside a generated launcher script, so a directive containing `;`, `|`, `&`, backticks, quotes or redirection is dropped rather than executed.
- **The rescue Hessian inherits the environment.** When `#rescue(GFN2-xTB/freq)` fires on a structure that failed to converge, the extra flags are reapplied to the Hessian run, so the frequencies are computed on the same potential energy surface as the optimization. Job type flags (`--opt`, `--md`, `--gfnff`, ...) are excluded, since the rescue run sets its own job type.

From the web generator. In **Advanced** mode (Section 3) the Geometry Optimization, Geometry Refinement and Energy Refinement blocks each grow an extra row as soon as their level of theory is set to standalone xTB: an **Implicit Solvent** model (ALPB, GBSA or CPCM-X) with its solvent list, and a free text **Extra xTB flags** field that takes a comma separated list without the leading dashes (`etemp 400, acc 0.5`). The generator writes them as the `#` directives above, filtering the reserved names and the unsafe characters listed here, and the template preview updates as you type, so it doubles as a check on what the command line will look like. Anything the controls do not cover can still be typed straight into the preview, or into the `.asc` file afterwards.

Understanding the Execution Pipeline

Upon parsing the `.asc` file and its embedded templates, ASCEC executes the stages in order. It begins with triplicated **Annealing** (`r3 --concurrent=3`) to generate raw candidates, then **preoptimizes** the whole pool with the GFN2-xTB `input1` template (`opt`) and applies the first **COSMIC** screening.

The surviving motifs are **refined** at `wB97X-D3BJ/def2-TZVPP` with frequencies (`ref input2`) and screened again, the `--th=opt` on this second `cosmic` re uses the threshold resolved by the first. Finally an **energy refinement** (`eref input3`) computes DLPNO-CCSD(T) single points on the DFT geometries, and a last **COSMIC** pass delivers the unique minima and their Boltzmann populations. Ending the protocol after the first or second `cosmic` yields the preliminary or rigorous description instead.

***Note:** From this point forward, the advanced sections of this manual explain each of these stages in detail: box sizing, the feature vector and thresholds, the templates, and how to run each function on its own.*

8 Box Size Suggestion

ASCEC anneals inside a hard walled cubic box of side L . Centers of mass are confined within the walls and any move that puts one outside is rejected. The choice of L matters: too small and the molecules are compressed, cannot rotate freely, and pick up artificial steric strain; too large and the search wastes its time in empty space.

One parameter controls it, the **packing fraction** ϕ , given as a percentage. It is the fraction of the box the system effectively occupies, and the box follows from it:

$$L = \left(\frac{V_{\text{eff}}}{\phi} \right)^{1/3} \quad (6)$$

Because the volume scales as L^3 , doubling the number of molecules grows L by only $2^{1/3} \approx 1.26$. A rule linear in the number of molecules would oversize the box badly, and increasingly so with system size.

8.1 How V_{eff} is Estimated

Each atom is treated as a hard sphere of its covalent radius; the spheres of one molecule are assembled in their real geometry and the **convex hull** of that surface gives the molecular volume V_{mol} . No shape is assumed: a single atom gives a sphere, a planar molecule a flattened polyhedron. The molecule’s **extent** E , the largest distance between any two atoms plus their radii, is its effective diameter whatever its orientation.

From these, ASCEC picks one of two routes automatically:

Method	Applies when	V_{eff}	Rationale
A	The system can form hydrogen bonds	$V_{\text{mol}}^{\text{total}} + V_{\text{HB}}^{\text{total}}$	Hydrogen bonds are directional and rigid, holding molecules in open arrangements. The extra volume accounts for the network they impose.
B	No hydrogen bond donors or acceptors	L_{diag}^3	With no directional constraint, the molecular extents laid end to end and read as a cube diagonal give a natural size.

Table 6: The two routes to the effective volume. Method A is preferred whenever it applies.

When Method B overestimates. Laying every molecular extent end to end is a deliberately generous upper bound: it assumes the fragments line up along one diagonal rather than packing together. For a few fragments that is a good approximation and the box is close to what the system needs. As the number of fragments grows, the sum of extents keeps growing linearly while a real cluster packs in three dimensions, so Method B returns a box noticeably larger than necessary. For a system with no hydrogen bonding and many fragments, treat the suggestion as an upper limit: check it against the visual test in the web generator and reduce it, or set L yourself, rather than annealing in empty space.

8.2 Choosing the Packing Fraction

The same ϕ means the same thing for water, methane, argon or a mixed system, because ASCEC computes V_{eff} and selects the method without intervention. The web generator’s presets set it for you: 30% for preliminary runs and 20% for rigorous and ultimate, the looser box giving molecules more room to reorganize during a longer search. Driving the command line directly, these ranges are useful guidance:

ϕ	Box	Use
10–15%	very loose	Isolated clusters; fragments explore independently before associating.
20%	loose	Cluster formation. The default for rigorous and ultimate runs.
30%	moderate	Quick surveys. The default for preliminary runs.
40%+	dense	Condensed phase or liquid like packing; rarely wanted for cluster work.

Table 7: Packing fraction guidance.

The command `ascec input.asc box` prints the analysis and the suggested length for several packing fractions without running anything else, and the web generator previews the box in three dimensions with a test that mimics the random rigid body moves, so an unsuitable box is visible before any time is spent on it.

9 Standalone Functions: Understanding Each Stage

Most users drive COSMIC-ASCEC through the automated pipeline of Section 10, where a single command chains annealing, optimization, clustering and refinement. This section takes that chain apart and runs each stage on its own. That is what you want when you need to inspect an intermediate result, recluster an existing set with different settings, or simply watch what the pipeline does on your behalf.

Two properties of standalone mode shape everything below. First, these commands **write input files and launcher scripts; they do not run the quantum chemistry themselves**. You run the launcher. Second, **sampling is stochastic**: each run starts from its own random seed, so the number of accepted configurations, and the number of motifs that survive clustering, differ from one run to the next. Repeating the example will not reproduce a particular count, and no count is quoted here for that reason.

The worked example is the water hexamer, and everything it needs ships with the software in `examples/w6/standalone/`: the input `w6.asc`, holding six water molecules and the annealing parameters of Block 1 (Section 7.2) with no protocol block and no embedded templates, and one template per calculation stage. Each template is shown where the stage that reads it is described.

Because those templates are separate files here rather than blocks embedded in the `.asc`, it is the **file extension** that tells ASCEC which program to write for: `.inp` for ORCA, `.com` for Gaussian, `.xtb` for standalone xTB. What goes inside them is the same either way, and is described in Section 7.3.1.

9.1 Annealing

Before launching, it is worth checking the box. The `box` subcommand prints the effective volume analysis of Section 8 and a table of recommended edge lengths, without running any quantum chemistry. This analysis depends only on the molecules you supplied, so unlike everything that follows it gives the same answer every time:

```
ascec w6.asc box
```

```
(base) :~/.../examples/w6/standalone$ ascec w6.asc box
```

Box length analysis

1. Molecular volume analysis:

```
-----
Number of molecules to place: 6
Total molecular hull volume: 10.50 Å³
Total H-bond network volume: 67.86 Å³
Effective volume: V_mol + V_HB = 78.36 Å³
```

3. Current box analysis:

```
-----
Cube's length = 5.00 Å
Current effective packing: 62.7%
+ Molecular: 8.4%, H-bond network: 54.3%
Current free volume: 47 Å³ (37.3%)
Largest molecular extent: 2.15 Å
```

Extremely dense - may cause steric clashes

4. Recommendation:

```
>>> Default suggestion: 7.3 Å (20% packing) <<<
```

- For isolated clusters: 8.1 Å (15% packing)
- For cluster formation: 7.3 Å (20% packing)
- For network studies: 6.8 Å (25% packing)

Use --box<P> to use a specific packing %
(e.g., --box10) for a 10%

The 5.0 Å box written into `w6.asc` is far too dense for this system, so we let ASCEC rewrite the edge length to a target packing fraction with the `--box` flag, and at the same time replicate the run with `r#` — here `r3`, for a triplicated run:

```
ascec w6.asc r3 --box20
```

```
Using recommended box size: 7.3 Å (20.0% effective packing)
Creating 3 replicated runs in 'annealing/'...
Created launcher script: launcher_ascec.sh
```

To run all simulations sequentially, use:
`./launcher_ascec.sh`

Nothing has been computed yet. The command wrote three input files, one per replica, and a script that runs them in series. Running that script is a separate act:

```
bash launcher_ascec.sh
```

Each replica starts from the “**Big Bang**” configuration: all centers of mass superimposed at the box center, the worst possible, and therefore least biased, starting point. ASCEC makes up to 100 random moves to find a first configuration whose energy is computable without atomic clashes, then anneals from there, writing the accepted structures, the energy trajectory (**tvse**), and the lowest energy structure to each replica folder for later inspection. If all 100 attempts fail, the run stops with diagnostics instead of a trajectory.

The launcher writes no energy plots. Each replica folder ends up with its **tvse_<seed>.dat** trajectory, but the diagrams of Figures 22 and 23 are produced by the replication stage of the automated pipeline, which the launcher bypasses. Generate them afterwards from the data that is already there:

```
ascec diagram
```

This writes **tvse_<seed>.png** beside every trajectory it finds and a combined **tvse_r3.png** for the replica group. The first accepted configurations sit far above the annealed ones and flatten everything after them into a single line; **ascec diagram --scaled** crops the energy axis to drop that initial descent and replots the same data on a readable scale.

```
/
├── annealing/
│   ├── tvse_r3.png
│   ├── w6_1/
│   ├── w6_2/
│   └── w6_3/
├── launcher_ascec.sh
└── w6.asc
```

9.2 Geometry Optimization

The annealing pool is large and highly redundant, so before any expensive refinement every accepted structure is relaxed with the fast semiempirical GFN2-xTB method [2]. This preoptimization must preserve the topology of the PES, letting each structure fall into its nearest local minimum without merging distinct basins or discarding valid low energy isomers.

The template that does it is **opt_xtb.inp**. It reaches GFN2-xTB through ORCA rather than through the xTB binary, which keeps one program in charge of the whole stage:

```
#name
! NATIVE-XTB2 Opt
%geom
maxiter 100
end
* xyz 0 1
#
*
```

NATIVE-XTB2 is ORCA’s own GFN2-xTB implementation, available from ORCA 6.1 onward and the reason no external xTB installation is needed here. On ORCA 5.x the keyword is **XTB2**

instead, and that route does call an external `xtb` binary. The two `#` placeholders are the ones described in Section 7.3.1: `#name` becomes a comment naming the structure, and the bare `#` between `* xyz 0 1` and the closing `*` is where the coordinates are injected.

The `opt` command scans for the annealing results, fills the template once per accepted structure, and writes the decks plus a launcher:

```
ascec opt opt_xtb.inp
```

With replicas present it first asks whether to treat each replica separately or to combine them into one numbered pool. Combining, answer `c`, is the usual choice: the replicas are independent samples of the same surface, and the clustering wants them in one pool. The decks land in `geometry_optimization/` as `opt_conf_1.inp`, `opt_conf_2.inp` and so on. If ORCA is on your `PATH` the launcher is generated for you, with the ORCA and OpenMPI environment already set; otherwise pass your own launcher as a second argument, `ascec opt opt_xtb.inp launcher_orca.sh`, and its environment block is reused. Without either, the decks are still written and only the launcher is missing.

```
/
├── annealing/
├── geometry_optimization/
│   ├── opt_conf_1.inp
│   ├── ...
│   ├── opt_conf_<N>.inp
│   └── launcher_orca.sh
├── w6.asc
└── opt_xtb.inp
```

```
cd geometry_optimization && bash launcher_orca.sh
```

9.3 Sorting the Results

The calculations leave one output file per structure scattered beside its input. The `sort` command groups each structure into its own subfolder, writes `combined_results.xyz` with the optimized geometries, records a wall time summary, and collects the output files into the `cosmic/` folder where the clustering will read them:

```
ascec sort
```

Use `--nosum` to skip the summary, or `--justsum` to write the summary without moving anything. The collected folder is named after the program and the number of structures in it, so an ORCA pool of `<N>` structures becomes `orca_out_<N>`:

```
/
├── annealing/
├── geometry_optimization/
├── cosmic/
│   ├── orca_out_<N>/
│   │   ├── opt_conf_1.out
│   │   ├── ...
│   │   └── opt_conf_<N>.out
└── w6.asc
```

9.4 COSMIC Clustering

With the outputs collected, the clustering runs from the `cosmic/` folder `sort` just created — the level *above* `orca_out_<N>/`, not inside it:

```
cd cosmic && cosmic -j4
```

COSMIC parses each output into the physicochemical feature vector of Equation 4, standardizes it, builds the UPGMA tree and cuts it at the resolved threshold, keeping the lowest energy structure of each cluster as its representative. The representatives get a folder of their own, `candidates_<NN>/`; `clustering_summary.txt`, the dendrograms and the per cluster data land beside it in `cosmic/`. Two arguments control the run:

- **--th (threshold):** the clustering granularity, the maximum distance in feature space for two structures to count as the same motif. The default, `--th=auto`, detects it per case; Section 6.3.7 explains how, and why the detected value is capped. Pass a number instead, as in `--th=2`, to set it yourself.
- **-j (jobs):** cores used to parse the output files. Omitted, COSMIC uses all of them.

A lower threshold creates tighter, more homogeneous clusters; a higher one yields broader families. Table 8 gives the ranges worth reaching for when you override the automatic choice.

Granularity	Value (τ)	Typical Use Case
Strict / Tight	0.5 - 1.5	Small/Rigid Systems. Distinguishes between specific rotamers or isoenergetic conformers with slight dihedral changes.
Standard	2.0 - 2.5	General Purpose. Default for molecular clusters (e.g., water hexamers). Balances distinct topology against vibrational noise.
Loose / Broad	> 3.0	Large/Flexible Systems. Groups broad structural families (e.g., folding patterns) while ignoring local side chain fluctuations.

Table 8: Recommended clustering threshold (τ) values.

Adding `--rmsd` enables the optional Step II geometric check inside each family. What the feature vector contains, how it is weighted and how the threshold is chosen are all covered in Section 6.3.

What the survivors are called. Every clustering pass promotes its input one rung up a three level ladder, and the name records the stage a structure came out of, so nothing has to be traced by the directory it was found in. This first pass reads unlabelled optimization output, so it emits the lowest rung:

Pass	Reads	Writes	What the set is
First	opt_conf_*.out	candidates_<NN>/	The distinct motifs present in the preoptimized pool.
Second	candidate*_opt.out	motifs_<NN>/	The same set after refinement at the higher level.
Third	motif*_opt.out	u_motifs_<NN>/	Unique motifs, after a final single point energy refinement.

Table 9: The three rungs of representatives. Each folder holds <label>_01.xyz, <label>_02.xyz and so on, ordered by energy, with the count in the folder name. The third rung needs an energy refinement stage, which exists in protocol mode only.

Caching. Parsing hundreds of quantum chemistry logs is COSMIC’s main bottleneck, so the parsed descriptors are cached to a `data_cache_XXXXXX.pkl` file on the first run. Reclustering at a different threshold then reuses the cache instantly, which is what makes standalone reclustering cheap; pass `--reprocess-files` (or delete the cache) whenever the underlying outputs change.

```

/
├── cosmic/
│   ├── orca_out_<N>/
│   ├── candidates_<NN>/
│   │   ├── candidate_01.xyz
│   │   ├── ...
│   │   └── all_candidates_combined.xyz
│   ├── dendrogram_images/
│   ├── extracted_clusters/
│   ├── extracted_data/
│   ├── clustering_summary.txt
│   └── data_cache_<hash>.pkl
└── w6.asc

```

9.5 Geometry Refinement

The representatives that survived clustering are the only structures worth spending real quantum chemistry on. The `ref` stage is the refinement counterpart of `opt`: it reads them and writes one high level input each. Run it from the `cosmic/` folder holding `candidates_<NN>/`, with the B3LYP template `ref_b3lyp.inp`:

```

#name
! B3LYP 6-31G
! Opt Freq
%pal
  nprocs 8
end
%geom
  maxiter 80
end
* xyz 0 1
#
*

```

```
ascec ref ref_b3lyp.inp
```

The decks are written to `geometry_refinement/`, each named after the representative it came from: `candidate_01_opt.inp`, `candidate_02_opt.inp` and so on, so a structure can be traced back through every stage by its name alone. As before, a launcher is generated when ORCA is on the PATH, and as before, running it is your step.

The same command accepts a Gaussian template instead. Handing it `ref_b3lyp.com` writes `.com` decks rather than `.inp` ones, with `!name` carrying the structure name and a bare `!` marking where the coordinates go, which are simply the Gaussian spellings of the two ORCA placeholders. One practical difference: launcher generation is automatic only for ORCA, so a Gaussian stage needs a launcher passed as the second argument.

```
!name
%nprocshared=8
%mem=4GB
#p B3LYP/6-31G Opt Freq

Water hexamer standalone refinement

0 1
!
```

Once the refinement finishes, the cycle closes as it did before: `ascec sort` to collect the outputs, which this time gathers them into `cosmic_2/`, then a second clustering pass over them:

```
cd cosmic_2 && cosmic --th=opt -j4
```

`--th=opt` reuses the threshold the first pass resolved, so the refined set is partitioned on the same footing as the preoptimized one instead of having a new threshold detected from a much smaller sample. This pass promotes the set one rung, writing `motifs_<NN>/` together with the Boltzmann populations of the refined structures. For most work that is where the chain ends; the third rung of Table 9 needs an energy refinement stage, which exists in protocol mode only.

9.6 Standalone Mode Scope

Every command in this section writes files. None of them runs a quantum chemistry program, and none of them judges what came back. That division is deliberate, and it is the whole of the difference between standalone and protocol mode:

- **Manual Calculations**
- **No failure is retried**

If you want any of that, use the protocol of Section 10. It runs these same stages in this same order, and adds between them the validation and repair that this section does without.

Standalone mode exists for the opposite purpose. Each command stops at a point where you can look: at the box analysis before committing to a run, at the trajectories before optimizing, at a generated deck before submitting a few hundred of them, at one clustering before trying another threshold on the same cached descriptors. It is the mode to reach for while you are learning how the method behaves or showing someone else, and the mode to leave behind once you want results you can trust without watching them being made.

10 Workflow Mode

Workflow mode runs a complete multistage protocol from a single `.asc` file. The protocol block and the embedded templates (Section 7.3.1) describe every stage; ASCEC then carries the structures from annealing through geometry optimization, COSMIC screening, geometry refinement and energy refinement, using **xTB** and **ORCA** for the quantum chemistry. Unlike standalone mode (Section 9), no files have to be filtered, renamed or resubmitted by hand between stages, and structures that fail to converge or retain imaginary frequencies are tried to corrected automatically (Section 10.5).

Workflow mode coordinates the calculations; it does not choose them. A poor level of theory, a wrong multiplicity or a broken template is carried faithfully through every stage, so the methods for each stage should be chosen and tested on the system first, with standalone runs, a preliminary protocol, and finally a review run (Section 10.2).

10.1 Pipeline Logic

Figure 19 shows the execution logic for a protocol with two refinement stages: annealing, pre-optimization and screening, then geometry and energy refinement, each followed by COSMIC, with the automatic feedback loops that send failed structures back for recalculation.

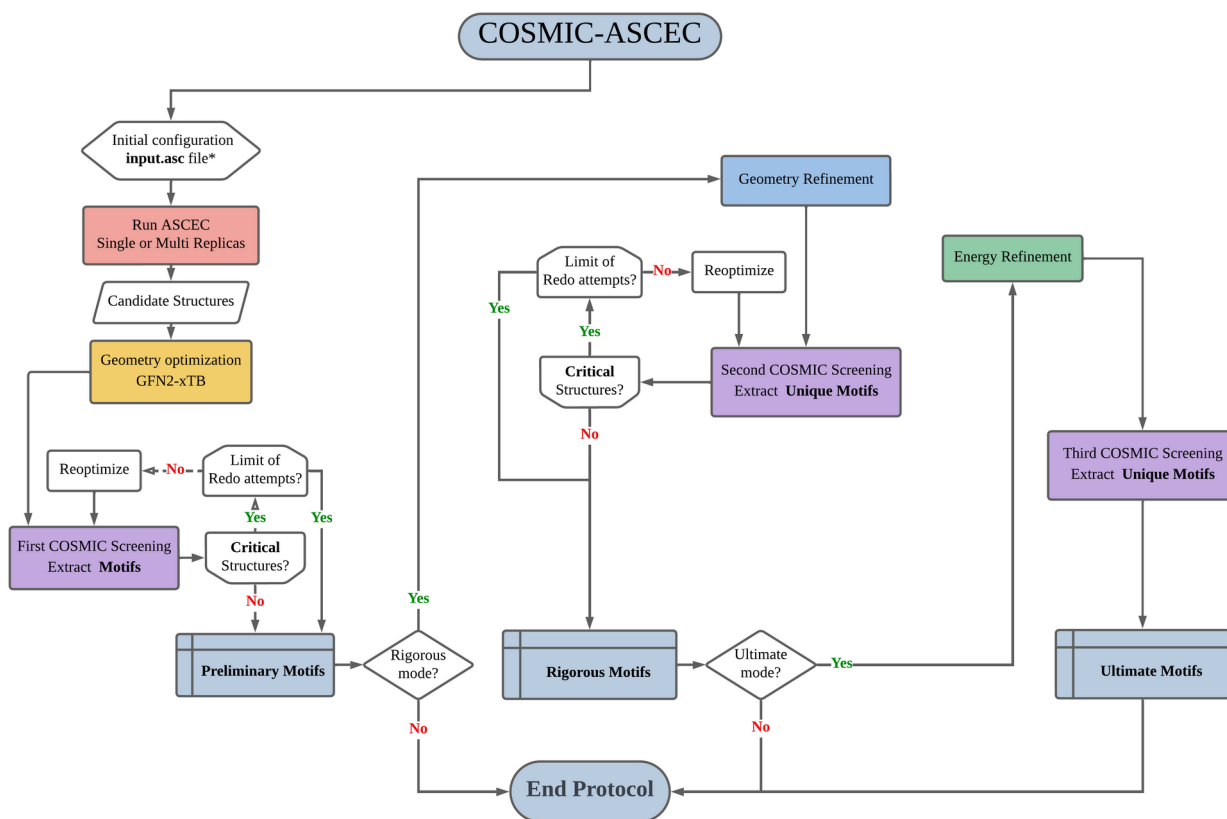


Figure 19: Schematic of the multistage protocol.

10.2 Review Runs: Rehearsing an Expensive Protocol

A rigorous or ultimate protocol can run for days or weeks, and almost all of that time goes to the `geometry_refinement` and `energy_refinement` stages (Figure 17), which run once per surviving structure. A mistake in an ORCA template, a wrong multiplicity, an unavailable method or a bad memory setting may therefore only appear after much of that time has been spent.

A **review run** tests the whole pipeline on a single structure. Adding `--review` to the protocol executes *every* stage exactly as written, but carries only one structure through the expensive stages: the lowest energy configuration found by the annealing, the **putative global minimum**. The whole protocol is rehearsed end to end for roughly the cost of one structure.

`--review` is a global modifier for a preliminary, rigorous or ultimate protocol (Section 1.1). Write it on its own line directly under the `.asc`, marker, or pass it on the command line (`ascec li5.asc --review`).

Example: the Li₅ cluster. The `li5` example shipped with the distribution is an open shell lithium pentamer (charge 0, multiplicity 2) whose ultimate protocol chains three quantum chemistry stages and three COSMIC screenings. Selecting *review* in the web generator adds the `--review` line and reduces the annealing to a single replica:

```
# Protocol

.asc,
--review,
r1,
opt --concurrent=8 input1,
cosmic -j4,
ref --concurrent=2 input2,
cosmic --th=opt -j4,
eref input3,
cosmic --th=opt -j4
```

The run is launched with the usual command:

```
ascec li5.asc
```

The stages then behave as follows:

- **Annealing runs in full.** The search itself is not reduced: each replica writes its complete set of accepted configurations. The web generator uses a single replica because only one structure continues past the annealing; if more replicas are written by hand they are all run, and the selection below is made across all of them.
- **One structure is selected.** The lowest energy accepted configuration across *all* replicas becomes the only input for the next stage. The run directory and file names are the same as in a full run (`opt_conf_<N>`, then `candidate_01`, `motif_01` and `u_motif_01` as each COSMIC pass reports it).
- **Every later stage sees one structure.** `opt`, `ref` and `eref` each run a single quantum chemistry job, and each COSMIC pass reports one input and one motif. Concurrency flags such as `--concurrent=8` are left in place and have nothing to parallelize.

For Li₅ the seven stages complete unattended in under an hour:

```

=== COSMIC ASCEC ===
-----
Progress [#####] 100.0%
-----
[1/7] annealing ✓
[2/7] geometry_optimization ✓
[3/7] cosmic (1→1) ✓
[4/7] geometry_refinement ✓
[5/7] cosmic_2 (1→1) ✓
[6/7] energy_refinement ✓
[7/7] cosmic_3 (1) ✓

Workflow finished
Start: 2026-09-23 11:35:35
End:   2026-09-23 12:31:08
Total wall time: 55m 32s
-----
Protocol summary saved to protocol_summary.txt

```

The useful part is the timing breakdown in `protocol_summary.txt`, because the full run scales with it:

Stage	Wall time	% total	Reading
Annealing	3m 01 s	5.4%	Fixed cost; does not grow with the number of motifs.
Optimization	3.8 s	0.1%	Semiempirical, negligible per structure.
Refinement	7m 01 s	12.6%	Cost of <i>one</i> wB97X-D3BJ/def2-TZVPP Opt+Freq.
Energy refinement	45m 26 s	81.8%	Cost of <i>one</i> DLPNO-CCSD(T1)/aug-cc-pVTZ point.

Table 10: Li₅ review run: where the time goes for a single structure.

Four fifths of the run is a single coupled cluster energy. That time, multiplied by the number of motifs the full run is expected to produce and divided by the concurrency, is the estimate to budget for (Section 10.2.1).

What a review run tells you: that the input file parses; that every embedded template produces a valid quantum chemistry input; that the requested methods and basis sets exist in the backend; that the charge and multiplicity are accepted; that each stage hands its output to the next; and roughly how long one structure takes in each stage.

What it does not tell you: anything about the potential energy surface. The selected structure is the lowest *at the annealing level of theory* and may not remain the global minimum after optimization and refinement. The motif counts are one by construction and say nothing about the diversity of the surface. It also does not test whether the requested concurrency fits the machine. A review run validates a protocol; it does not sample a surface, and its summary file is labelled accordingly.

Promoting to the full run. Once the review run completes cleanly, delete the `--review` line, restore the number of replicas wanted for the search, and launch the same file again.

10.2.1 Review Run versus Full Run: the Li₅ Cluster

Li₅ is small enough to run twice from the same `li5.asc`, once with `--review` and once in full, which shows how well a review run predicts the real cost. Removing the `--review` line and restoring three replicas (`r3 --concurrent=3`) gives the complete ultimate protocol (Table 11), which ends with four unique motifs.

Stage	Structures	Wall time	Notes
Annealing	192 accepted	2 m 38 s	3 replicas, run concurrently
Optimization	192/192	13.9 s	GFN2-xTB, 8 concurrent
COSMIC	192 → 12	—	candidates
Refinement	12/12	56 m 43 s	mean 7 m 31 s per structure, 2 concurrent
COSMIC	12 → 4	—	motifs; one redo cleared 8.3% critical
Energy refinement	4/4	3 h 14 m 21 s	mean 48 m 34 s per structure
COSMIC	4	—	unique motifs
Total		4 h 13 m 57 s	against 55 m 32 s for the review run

Table 11: Li₅, full ultimate protocol. Compare with Table 10.

How well did the review run predict it? The per structure costs carry over almost exactly: 7 m 01 s for one geometry refinement against a mean of 7 m 31 s in the full run, and 45 m 26 s for one energy refinement against a mean of 48 m 34 s. What the review run cannot supply is the number of motifs; that estimate has to come from a cheap preliminary run.

The shape of the cost carries over too. In the review run 82% of the time was a single coupled cluster energy, and in the full run energy refinement took 76.5%. A stage that dominates the review run will dominate the full run, so that stage is where a wrong keyword or an unaffordable `%maxcore` is most worth catching.

Here the putative global minimum picked from the annealing also survived optimization and refinement, because the lowest semiempirical structure was also the lowest at the coupled cluster level. This is not guaranteed: on larger or more flexible systems the energy ordering often changes between levels of theory, which is why the full run is needed. The structure from a review run is a first guess, occasionally, as here, a very good one.

10.3 Worked Example: Formic Acid Dimer

This example runs a full rigorous protocol on the formic acid dimer, the step that follows a successful review run. It is also a test of conformational sampling: each monomer can adopt a *syn* (*cis*) or *anti* arrangement of its HC–OH bond, so three dimer families, *aa*, *as* and *ss*, are possible. The literature explored each family separately [14]; here a single run starts from two superimposed *syn* monomers. The workflow runs the annealing and preoptimization with GFN2-xTB, then refines the COSMIC motifs at wB97X-D3BJ/def2-TZVPP with ORCA. The file below, with its parameters, coordinates, protocol and two embedded templates, is the only input required.

```

# Input for cis_formic Simulation

1 10          # Line 1: Simulation Mode & Number of Config
6.7          # Line 2: Simulation Cube Length (Angstroms)
2            # Line 3: Annealing Quenching route
100.0 10.0 50 # Line 4: Linear Quenching Parameters
500.0 5.0 100 # Line 5: Geometric Quenching Parameters
3000 10       # Line 6: Max Monte Carlo Cycles per T and floor value
1.0 1.0       # Line 7: Maximum Displacement (A) & Rotation (radians)
60 120        # Line 8: Conformational sampling (%) & Max dihedral (deg)
3 xtb         # Line 9: QM Program Index & alias (1:Gaussian 2:ORCA 3:xtb)
GFN2-xTB      # Line 10: Hamiltonian & Basis Set
1 4           # Line 11: nprocs (QM calculations and ASCEC evaluation)
0 1           # Line 12: Charge & Spin Multiplicity
2            # Line 13: Number of Molecules (nmo)

# Lines below: Molecule Definition

*
5
cis1
C    0.340460   -0.288340   0.000000
O    1.570760   -0.288340   0.000000
O    -0.288840   0.913960   0.000000
H    -0.369640  -1.132440   0.000000
H    -1.252740   0.795160   0.000000
*
5
cis2
C    0.340460   -0.288340   0.000000
O    1.570760   -0.288340   0.000000
O    -0.288840   0.913960   0.000000
H    -0.369640  -1.132440   0.000000
H    -1.252740   0.795160   0.000000
*

# Protocol
# (comma = continue to next stage, dot = pause at breakpoint)

.asc,
--maxprint,
r5 --box20 --concurrent=5,
opt --concurrent=8 input1,
cosmic -j4,
ref --concurrent=2 input2,
cosmic --th=opt -j4

```

```

# Embedded QM templates

#xtb input1
#name
#rescue(GFN2-xTB/freq)
# GFN2-xTB Opt Tight
# nprocs 1
# maxiter 200

#orca input2
#name
#rescue(HF-3c/freq)
! wB97X-D3BJ def2-TZVPP def2/J RIJCOSX DEFGRID3
! Opt Freq TightOpt TightSCF
%pal
  nprocs 12
end
%geom
  maxiter 80
end
* xyz 0 1
#
*
```

10.3.1 Resource Considerations

The number of cores a stage uses is the number of concurrent jobs times the cores per job, and it must fit the machine. If a stage asks for more cores or memory than are available, the calculations either fail or compete for the same cores and slow down. A review run cannot catch this, since it runs one structure per stage and the concurrency flags have nothing to parallelize.

This example ran on a 24 core Intel Xeon workstation:

- **Annealing:** five replicas run concurrently (`r5 --concurrent=5`), each QM evaluation on 1 core (Line 11).
- **Optimization:** up to 8 concurrent GFN2-xTB jobs on 1 core each (`opt --concurrent=8, # nprocs 1`).
- **Refinement:** 2 concurrent ORCA jobs on 12 cores each (`ref --concurrent=2, nprocs 12`), which uses all 24 cores on the most demanding stage.

10.3.2 Running the Protocol

If the `.asc` file contains a protocol block, ASCEC runs it directly:

```
ascec cis_formic.asc &
```

`ascec cis_formic.asc protocol` is equivalent; the `protocol` keyword is only required to restart from a given stage (Section 10.4). Detaching a running job, following it with `ascec status`, and queuing a second protocol with `after` are described in the water hexamer example (Section 5.8).

10.3.3 Outputs

Starting from the single input file, the finished rigorous protocol leaves one directory per stage and the final results in the working directory:

```

/
├── annealing/
├── geometry_optimization/
├── cosmic/
├── geometry_refinement/
├── cosmic_2/
├── boltzmann_distribution.txt
├── cis_formic.asc
├── final_ensemble.mol
├── final_ensemble.xyz
├── protocol_670892.pkl
└── protocol_summary.txt

```

`protocol_summary.txt` records the pipeline and its timing. The run took 10 h 42 m 34 s: 0.14% in the annealing, 0.15% in the GFN2-xTB optimization and 99.71% in the DFT geometry refinement (Figure 17). The timing breakdown of the summary:

Stage	Duration	% Total
-----	-----	-----
Annealing	0:00:55.445	0.1%
Optimization	0:00:58.358	0.2%
Refinement	10:40:38.651	99.7%

`final_ensemble.xyz` and `final_ensemble.mol` hold the final motifs as a single trajectory, ranked by Boltzmann population (`boltzmann_distribution.txt`). The first structure is the putative global minimum.

10.3.4 Results

Five replicas accepted 574 candidates; the first COSMIC pass reduced them to 46 motifs and the final pass, after refinement and three redo attempts (Section 10.5.2), to **21 unique motifs** (Figure 21): 7 *aa*, 10 *as* and 4 *ss* dimers, with one visual redundancy. Eighteen coincide with the reference MP2 structures, three reference structures were missed and two new motifs were found. Although the search started from two superimposed *syn* monomers (Figure 20), it recovered all three families, crossing the monomer isomerization barrier during the stochastic search.

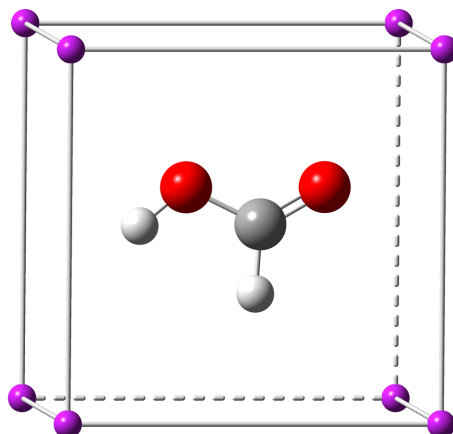


Figure 20: Initial simulation box: two superimposed *syn* formic acid monomers.

The Boltzmann distribution at 298.15 K is dominated by the cyclic *aa* dimer with its double hydrogen bond, which holds 99.99% of the population. The next motif lies 5.46 kcal/mol higher, far above $RT \approx 0.59$ kcal/mol, so every other conformer is effectively inaccessible at room temperature. The files behind these numbers are described in Section 11.

10.4 Execution Control and Resumption

10.4.1 Stage Delimiters

- **Comma (,):** continue to the next stage as soon as this one finishes.
- **Dot (.):** breakpoint. The workflow finishes this stage and stops, so the intermediate results (for example, the number of motifs COSMIC kept) can be inspected before the next stage is launched.

Excluding structures at a breakpoint. If the inspection shows structures that should not go on (a dissociated cluster, a duplicate the threshold missed), they can be removed from the input of the next optimization or refinement stage before resuming. Structures are selected by number, singly or as ranges:

```
ascec cis_formic.asc exclude ref 3,7-9
```

excludes `motif_03` and `motif_07` to `motif_09` from the refinement stage; `opt` and `opt2` select `opt_conf_N` in the first and second optimization stages. `ascec cis_formic.asc exclude` alone lists the current exclusions, and `exclude ref clear` (or `exclude clear` for all stages) removes them. Exclusions are stored in the protocol cache and take effect when the workflow is resumed.

10.4.2 Resuming the Workflow

Progress is recorded in a cache file (`protocol_<id>.pk1`). After a breakpoint or an interruption, reissuing the same command resumes from the first unfinished stage; completed stages are read from the cache and are not recalculated:

```
ascec cis_formic.asc
```

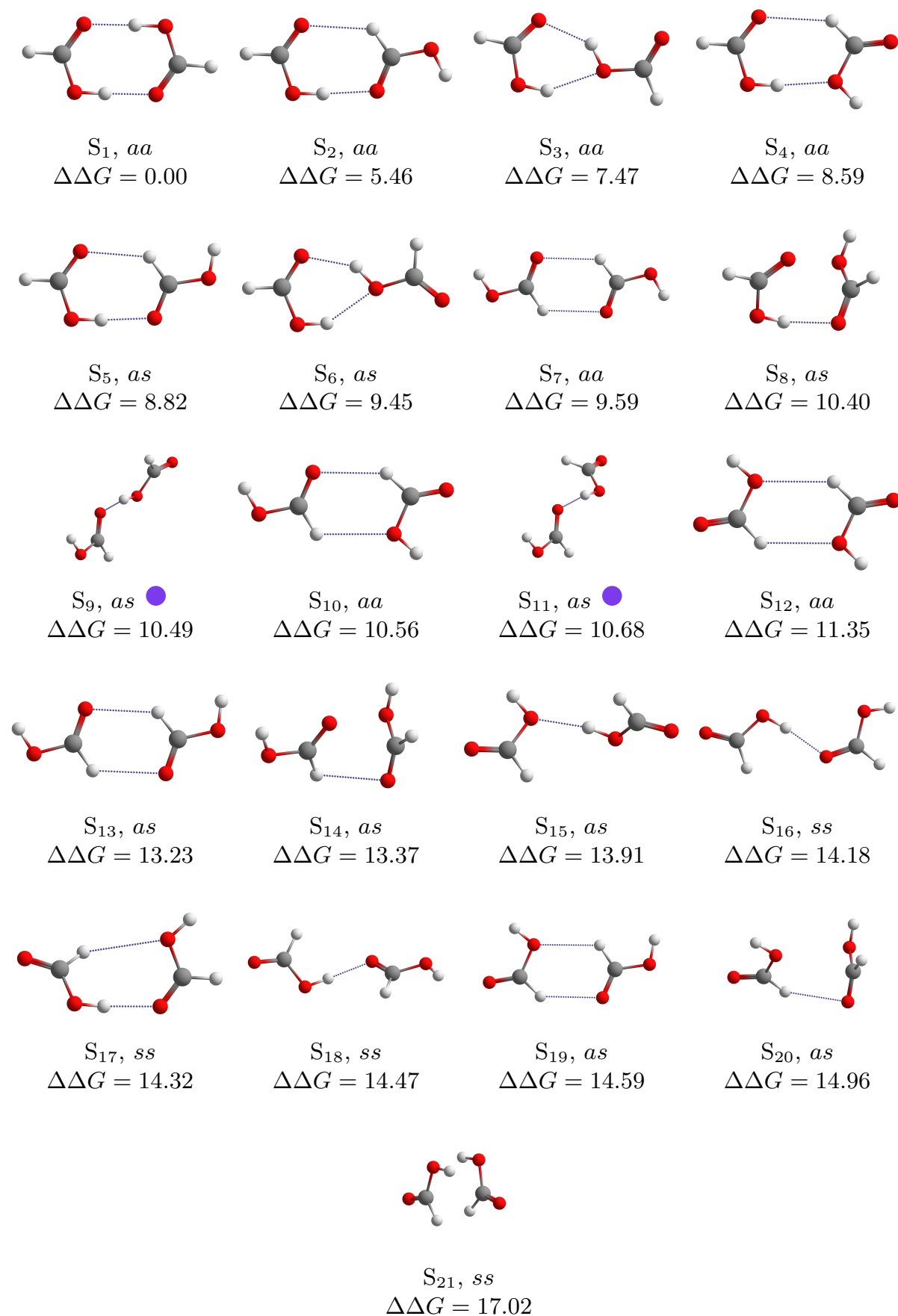


Figure 21: Formic acid dimer: minima from the rigorous protocol, with composite energies in kcal/mol.

10.4.3 Restarting from a Stage

A stage is selected by its position in the protocol, the [k/N] number shown on the progress screen. For the formic acid protocol:

```
1 annealing → 2 geometry_optimization → 3 cosmic → 4 geometry_refinement → 5
                                     cosmic_2
```

The optimization and refinement stages can also be named: **opt** (or **opt1**, **opt2**, ...) and **ref** (or **ref1**, **ref2**, ...).

Restart from scratch. Rerun stage 2 and everything after it:

```
ascec cis_formic.asc protocol 2
```

Warning: this **deletes** the stage directories and cache entries from stage 2 onwards. Completed calculations in those stages are lost.

Continue an incomplete stage. Mark stage 2 as unfinished without deleting anything, so the workflow runs only the calculations that are still missing:

```
ascec cis_formic.asc protocol 2 -i
```

10.5 Failure Recovery

Two mechanisms handle calculations that go wrong. Neither can fix a systematic error such as invalid template syntax; templates must be tested before a long run.

10.5.1 Retry: Launch Failures

A calculation that dies within its first few seconds (a crashed launch, a busy file system, a transient error from the QM program) is relaunched automatically, up to 10 times. The limit is fixed; the progress line reports how many attempts a job needed, e.g. **Running: opt_conf_1.inp...** ✓ (2nd Attempt).

10.5.2 Redo: Convergence and Topology

The final ensemble must contain only **true local minima**. The COSMIC pass after each optimization or refinement stage checks every structure, and the redo loop sends failed structures back for recalculation:

- **Not converged** (maximum iterations reached): the calculation is resubmitted from its last geometry.
- **One imaginary frequency:** a first order saddle point or a soft mode. The geometry is displaced along the imaginary mode, toward the neighbouring minimum, and reoptimized.
- **Several imaginary frequencies:** usually a numerical convergence artifact. The geometry is displaced along the largest imaginary mode and reoptimized.

If a structure still fails, the method given in the template's **#rescue(...)** line (Table 4) is used to reoptimize it.

COSMIC sorts the structures that are not clean minima into two sets:

- **Skipped:** every structure with imaginary frequencies or without a converged result.

- **Critical:** the skipped structures that match no existing cluster. They may be a motif that would be lost if they were discarded. A skipped structure that does match a cluster of true minima, compared on the reduced feature vector (Section 6.3.5), is already represented; it is kept in that cluster as redundant and does not count as critical.

A redo is triggered when the critical fraction exceeds its threshold (`--critical`, default 0%, so every critical structure is retried) or, if `--skipped=N` is given, when the skipped fraction exceeds $N\%$. The loop repeats until the threshold is met or the limit of attempts is reached (`--redo=N`, default 3). These flags, like `--concurrent`, are written on the `opt` or `ref` line and act on the COSMIC pass that follows it.

Cost. Each redo attempt is another round of the stage’s most expensive calculation. The loop is meant to recover soft modes and saddle points, not to make up for an unsuitable method. Only critical structures are resubmitted, which keeps the cost down: structures already represented by a true minimum are not recalculated, while potential missing minima, which would leave gaps in the PES, are.

Example: the formic acid run. The first screening (GFN2-xTB, 574 $\rightarrow$ 46 motifs) passed with 0% critical and needed no redo. The DFT surface was stricter: the final screening flagged 11 of the 46 refined structures, and `protocol_summary.txt` records how the loop resolved them:

```
[5/5] cosmic ✓
Working dir:      /cosmic_2/orca_out_46
Threshold:       opt
Unique Motifs:    21 representatives
Initial validation
Critical:         23.9% (11 structures)
Skipped:         23.9% (11 structures)
Target: critical ≤ 0% | Actual: 23.9%
Final validation (3 Redo Attempts)
Critical:         0.0% (0 structures)
Skipped:         0.0% (0 structures)
Target: critical ≤ 0% | Actual: 0.0%
```

Over the three attempts, 4 of the 11 structures converged to true minima. The other 7 never converged within the iteration limit, but each matched an existing cluster on the reduced feature vector, so they are reported as redundant (the *reduced matched to full* count of the clustering summary) and add no motif of their own. A structure that still matched nothing after the last attempt would stay critical and be set aside in `skipped_structures/` (Section 11.4.4).

11 Output Files

This section walks through the files of a finished protocol, using the formic acid dimer rigorous run of Section 10.3 as the reference. Every excerpt below is taken from that run, which is part of the Zenodo deposit (Section 12); only the numbers change from system to system.

11.1 Print Levels: miniprint and maxprint

A workflow runs at one of two print levels:

- **Miniprint** (default): at the end of the run, the per structure job folders of each QM stage are removed. Only the files needed to interpret and reproduce the results are kept.
- **Maxprint** (`--maxprint`): every file of every stage is kept. The formic acid reference run, with 574 xTB and 46 ORCA jobs, occupies about 555 MB. A review run (`--review`, Section 10.2) always uses maxprint, since its purpose is to be inspected.

After a miniprint rigorous run, the project directory contains:

```

/
├── annealing/ .....untouched
├── geometry_optimization/
│   ├── combined_results.xyz | .mol
│   ├── xtb_summary.txt
│   └── geom_opt_out/ .....job files of the motifs in the final ensemble
├── cosmic/ .....untouched
├── geometry_refinement/
│   ├── combined_results.xyz | .mol
│   ├── orca_summary.txt
│   └── geom_ref_out/ .....job files of the final motifs
├── cosmic_2/ .....untouched
├── boltzmann_distribution.txt
├── cis_formic.asc
├── final_ensemble.xyz | .mol
├── protocol_<id>.pkl .....kept for resuming
└── protocol_summary.txt

```

Miniprint never removes anything from `annealing/` or from the `cosmic*` folders. An ultimate run adds `energy_refinement/` (with `energy_ref_out/`) and `cosmic_3/`. A preliminary run, whose only QM stage has no frequencies, writes `possible_final_ensemble.*` instead of `final_ensemble.*` and has no Boltzmann file. Under maxprint, the stage folders keep every job folder (`opt_conf_N/`, `motif_NN/`) instead of the `*_out/` subsets.

11.2 Annealing Output

The `annealing/` folder holds one subfolder per replica, named after the input (`cis_formic_1/` to `cis_formic_5/`), and the combined energy plot of all replicas (`tvse_r5.png`). Each replica folder contains:

```

cis_formic_4/
├── cis_formic_4.asc .....input of this replica
├── cis_formic_4.out .....main log
├── result_567430.xyz | .mol .....all accepted configurations
├── resultbox_567430.xyz | .mol .....same, with the box drawn
├── rless_567430.out .....lowest energy configuration
├── tvse_567430.dat | .png .....energy evolution
└── anneal.out | .xyz .....last QM single point

```

11.2.1 Main Log File

The log (<name>_<k>.out) records the system, the parameters and the complete history of the annealing. It opens with a description of the system and the settings read from the input:

```
Elemental composition of the system:
  H   4
  C   2
  O   4
There are a total of 10 nuclei

Cube's length = 6.70 A
Box suggestion (Method A): 7.3 / 6.7 / 6.2 A (15/20/25%)
...
Conformational sampling: 60.0% probability, max dihedral rotation =  $\pm 120.0^\circ$ 

Energy calculated with Xtb
Hamiltonian: GFN2-xTB
Charge = 0   Multiplicity = 1

Seed = 567430
Geometrical quenching route.
  To = 500.0 K  %dism = 5.0 %  nT = 100 steps
=====
History: [T(K), E(u.a.), n-eval, Criterion]

    500.00   -22.528967         0      Intv
    500.00   -22.532246         3      LwE
    475.00   -22.523703         1      Mpol
```

Each history line is an accepted configuration: temperature, energy, the number of energy evaluations since the previous line, and the acceptance criterion: **Intv** for the initial configuration, **LwE** for a move that lowered the energy, and **Mpol** (or **EMpol**) for a move accepted by the Metropolis criterion.

A run that completed without errors ends with the termination flag and a summary, which **tail** shows at once:

```
tail -n 13 cis_formic_4.out
```

```
** Normal annealing termination **
Total Wall time: 0 days 0 h 0 min 34 s 128 ms
Energy was evaluated 783 times

Energy evolution in tvse_567430.dat
Configurations accepted by Max.-Boltz. statistics = 69
Accepted lower energy configurations = 62
Accepted configurations in result_567430.xyz = 131
Lowest energy configuration in rless_567430.out
Lowest energy = -22.56275217 u.a. (Config. 130)
```

11.2.2 Reproducibility and Metadata

Each annealing run is initialized with a six digit random seed, which seeds all internal pseudorandom number generators and keeps parallel replicas statistically independent. The seed is written to the log and appended to the names of the output files (`result_567430.xyz`, `tvse_567430.dat`, ...). Rerunning ASCEC with the same seed and input reproduces the same sequence of proposed moves. Reproducing the full trajectory also requires an identical software environment, meaning the same Python and NumPy versions and the same version, build and parallel settings of the QM program. Even small numerical differences in the energies can change an accept or reject decision and make the trajectories diverge.

Setting the seed. The seed is drawn at random by default, but it can be fixed from the environment. `COSMIC_ASCEC_SEED` sets a single seed; `COSMIC_ASCEC_SEEDS` takes a comma separated list with one seed per replica, so a whole replicated protocol can be reproduced from one command:

```
COSMIC_ASCEC_SEED=878186 ascec glyw2.asc protocol
```

```
COSMIC_ASCEC_SEEDS=871246,112412,261557 ascec li5.asc protocol
```

The number of seeds in `COSMIC_ASCEC_SEEDS` must equal the number of replicas (three for the `li5` example); a mismatch is reported as an error.

11.2.3 Simulation Generated Data

I. Lowest Energy Configuration (`rless_<seed>.out`)

The Cartesian coordinates of the lowest energy configuration visited during the run, molecule by molecule, with the names given in the input:

```
# Configuration: 130 | Energy = -22.56275217 u.a.
5
cis1
C      -1.365907      0.202059      2.652191
(...)
```

II. Energy Evolution Log (`tvse_<seed>.dat`)

One line per accepted configuration, in three columns: the **cumulative number of energy evaluations**, the **temperature (K)** and the **energy (Hartree)**:

# n-eval (Cuml)	T(K)	E(u.a.)
1	500.00	-22.528967
4	500.00	-22.532246
5	475.00	-22.523703

These files are the source of the energy profile of each replica (Figure 22) and of the combined plot of all replicas (Figure 23). Both are produced automatically; to regenerate them, for example with an autoscaled energy axis, run:

```
ascec diagram --scaled
```

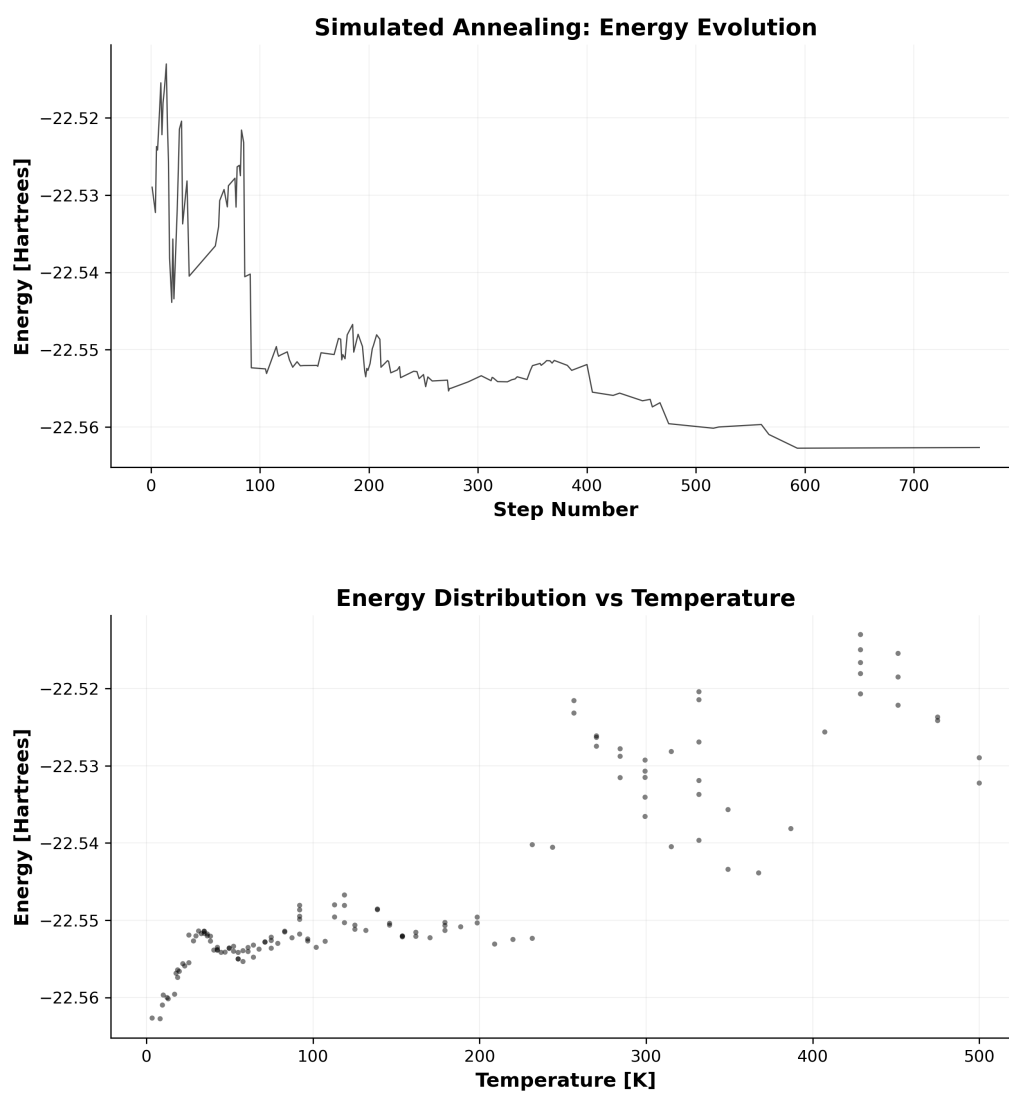


Figure 22: Energy evolution of one formic acid replica (`tvse_567430.png`).

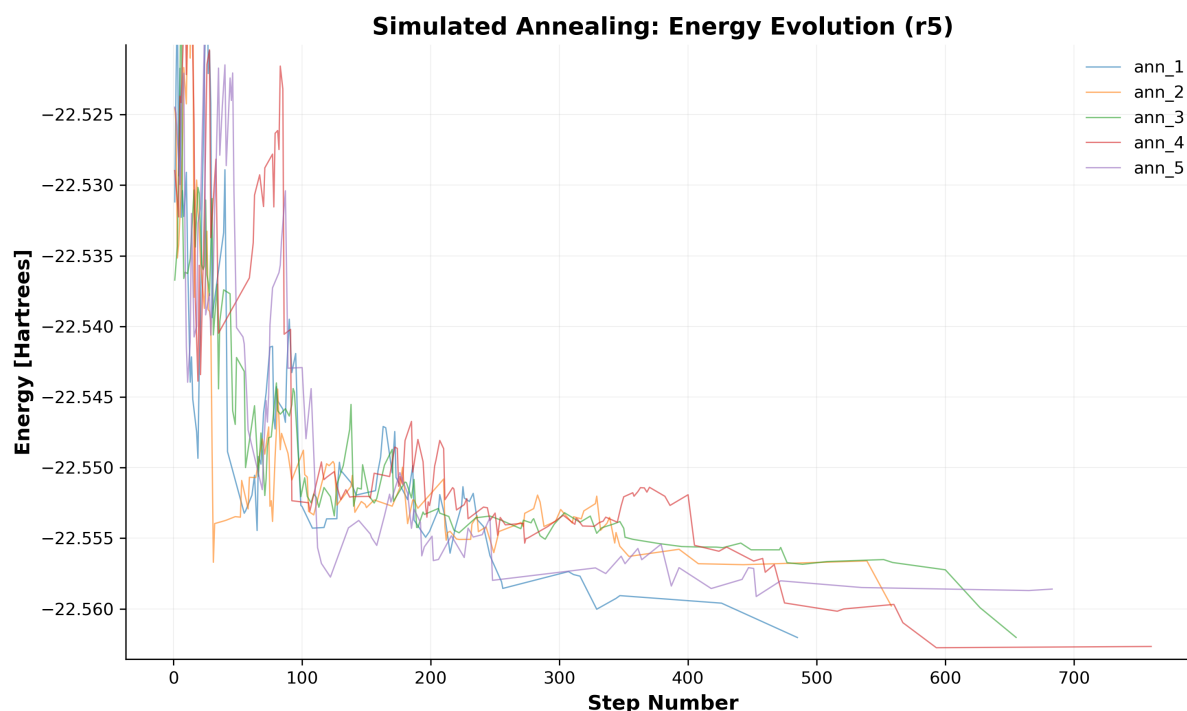


Figure 23: Energy evolution of the five formic acid replicas (`tvse_r5.png`).

III. Structural Trajectory (`result_<seed>.xyz` | `.mol`)

All accepted configurations in order, ready to animate in a molecular viewer (Avogadro2, Molekel, IQmol, GaussView). Each frame records its energy and temperature:

```
10
Configuration: 1 | E = -22.52896735 a.u. | T = 500.0 K
C      3.068773      2.767947      0.347586
O      3.589632      2.814281     -0.766056
(...)
```

IV. Simulation Box Visualization (`resultbox_<seed>.xyz` | `.mol`)

The same trajectory with the eight corners of the simulation box added as dummy atoms labelled X, to check the confinement visually. The `--nobox` flag skips these files.

```
18
Configuration: 1 | E = -22.52896735 a.u. | BoxL=6.7 A (X box markers)
C      3.068773      2.767947      0.347586
(...)
X     -3.350000     -3.350000     -3.350000
X      3.350000     -3.350000     -3.350000
(...)
```

Avogadro2 renders the box from either format; IQmol and GaussView need the `.mol` file.

11.3 Calculation Output

Each QM stage writes its own folder: `geometry_optimization/`, `geometry_refinement/` and, in ultimate runs, `energy_refinement/`. They share the same layout:

- **Job folders.** One per structure: `opt_conf_N/` in the optimization stage (numbered across the pooled replicas) and `motif_NN/` in the refinement stages (numbered after the COSMIC motifs). Each folder holds the complete input and output of that job: for xTB, the `.out`, the optimized `.xtbopt.xyz`, charges, bond orders and restart files; for ORCA, the `.inp`, `.out`, `.gbw`, `.hess`, the trajectory `_trj.xyz` and the property file. A structure that went through the redo loop also keeps its previous output (`.out.backup`) and, if the rescue method was used, the `_rescue.inp` and `_rescue.hess` files. Under miniprint, only the folders of the final motifs are kept, in `geom_opt_out/` or `geom_ref_out/`.
- **Combined geometries** (`combined_results.xyz` | `.mol`). The optimized geometry of every structure of the stage in one file, for visualization. In the reference runs, made with an earlier version, the file of the optimization stage still holds the input geometries (identical to `combined_r5/`); current versions collect the xTB optimized ones:

```
10
Coordinates from ORCA-job motif_01_opt E -379.797397521675
(...)
```

- **Execution summary** (`xtb_summary.txt`, `orca_summary.txt`). Timing and convergence statistics for the stage, followed by every job with its energy, number of cycles and time. The statistics block:

```
=====
Summary of all calculations:
  Number of jobs: 46
-----
  Total execution time: 14:41:27.672
  Mean execution time: 0:19:9.732
  Shortest execution time: 0:6:55.481
  Longest execution time: 0:34:15.936
  Total wall time: 10 hours, 39 minutes
-----
  Mean cycles: 34
  Min cycles: 10
  Max cycles: 70
  Non-converged: 7
=====
```

The seven non-converged jobs are the ones the redo loop could not recover (Section 10.5.2).

- **Launcher** (`launcher_xtb.sh`, `launcher_orca.sh`). The shell script that ran the jobs, with the program environment already set.
- **Pooled annealing input** (`combined_r5/`, **optimization stage only**). The accepted configurations of all replicas merged into one numbered pool, from which the `opt_conf_N` inputs were generated.

11.4 COSMIC Output

Each COSMIC pass writes its own folder: `cosmic/` after the optimization, then `cosmic_2/`, `cosmic_3/` after each refinement. The final pass of the formic acid run contains:

```
cosmic_2/
├── orca_out_46/ ..... copies of the 46 outputs that were clustered
├── umotifs_21/ ..... one representative per cluster
│   ├── umotif_01.xyz ... umotif_21.xyz
│   ├── all_umotifs_combined.xyz | .mol
│   └── umotifs_dendrogram.png
├── dendrogram_images/
│   ├── dendrogram.png
│   └── threshold_diagnostic.png
├── extracted_clusters/ ..... the members of each cluster
├── extracted_data/ ..... descriptor statistics per cluster
├── boltzmann_distribution.txt
├── clustering_summary.txt
└── data_cache_010568.pkl ..... parsed descriptors
```

The first pass has the same layout, with `xtb_out_574/` and `motifs_46/` (each file name records the configuration its motif came from, e.g. `motif_01_opt_conf_378.xyz`) and no Boltzmann file, since the xTB optimization has no frequencies. When structures remain critical after the redo loop, a `skipped_structures/` folder is added (Section 11.4.4).

11.4.1 Threshold Diagnostic

`threshold_diagnostic.png` shows how the clustering threshold τ was chosen (Section 6.3.7). The merge heights of the dendrogram are sorted and plotted; the knee of the curve, where the heights start to rise steeply, separates merges between near duplicates from merges between genuinely different structures. The legend compares the applied cut with the standard $\tau = 2.0$ and the Mojena criterion, giving for each the number of clusters n_c and the similarity floor it implies. For the formic acid preoptimization (Figure 24), the knee gives $\tau = 0.66$ and 46 clusters, with a similarity floor of 98.2%; the standard threshold would have merged them into 19.

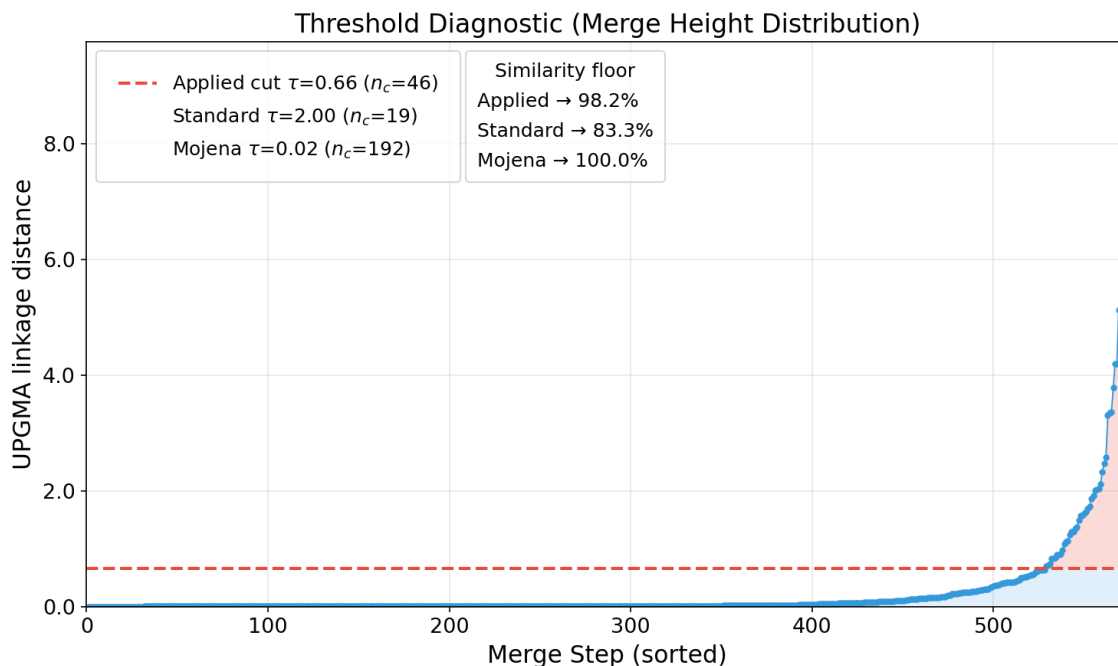


Figure 24: Threshold diagnostic of the first COSMIC pass (574 GFN2-xTB structures).

11.4.2 Dendrograms

The dendrogram (`dendrogram.png`) is the main tool for reading a clustering. It is a hierarchical tree whose vertical axis is the UPGMA linkage distance between the physicochemical feature vectors, with the applied τ drawn as a red dashed line. Branches joined below the line form one cluster; branches that only join above it are distinct motifs. Figure 25 shows the final pass of the formic acid run:

- **Redundancy.** Configurations 21, 04, 01 and 20 join below $d \approx 0.1$: they are the same structure, reached from different starting points. Together they form cluster 1, the cyclic *aa* global minimum.
- **Close but distinct.** Configurations 34 and 33 join at $d \approx 0.58$, just below $\tau = 0.66$, and are grouped. Configurations 22 and 25 join at $d \approx 0.85$, just above it, and are kept as separate motifs.
- **Global separation.** The global minimum family joins the rest of the tree only at the top ($d \approx 8.6$): its double hydrogen bond makes it the most distinct structure of the surface.

The tree has 39 leaves for 46 structures. The seven structures that never converged have no frequencies, so they cannot enter the tree on the full feature vector; they are assigned to their clusters afterwards on the reduced vector (Section 6.3.5).

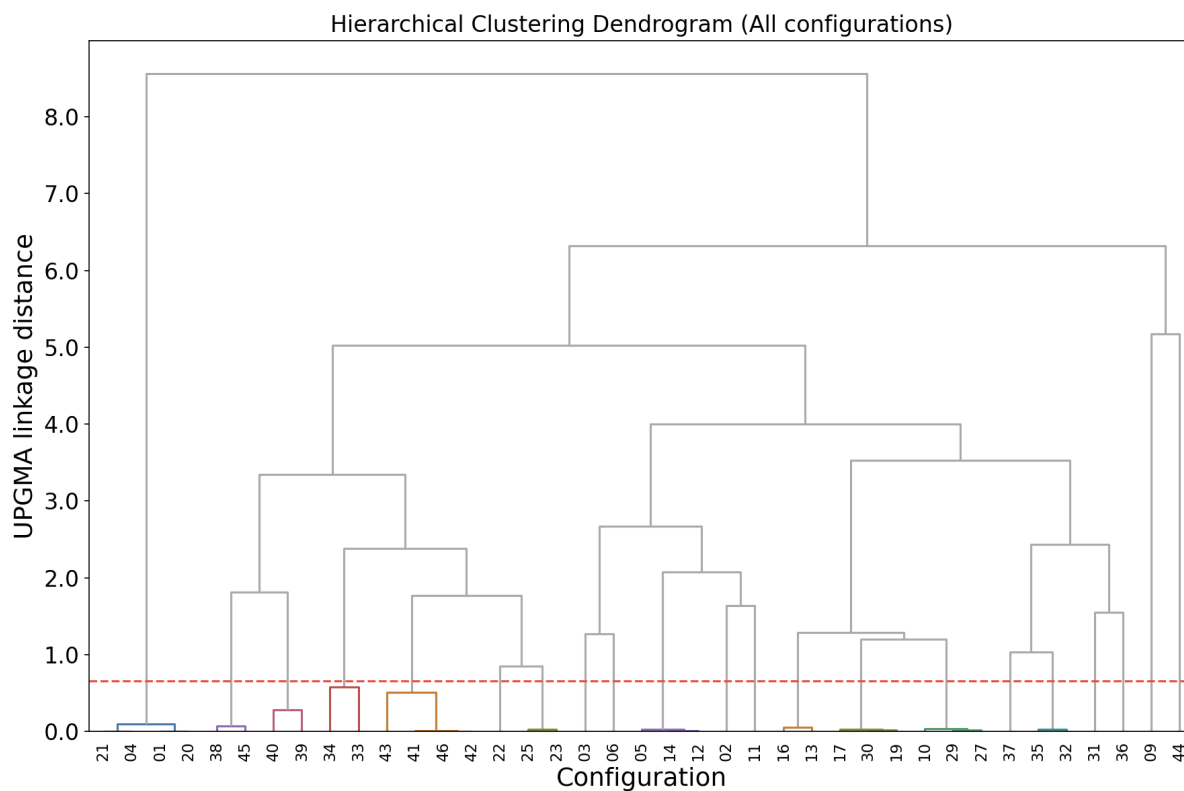


Figure 25: Dendrogram of the final COSMIC pass of the formic acid run (`cosmic_2`, $\tau = 0.66$).

11.4.3 Clustering Summary Log

`clustering_summary.txt` opens with the threshold and the global statistics of the pass:

```
Clustering Results for: orca_out_46
COSMIC threshold: tau = 0.6628 (user)
  details: tau = 0.6628, r >= 0.9854, N_f = 15.00, d_med = 4.8498, source = user
Total configurations processed: 46
Total files skipped: 0 (0.0%)
Critical skipped files: 0 (0.0%)
Reduced matched to full (redundant, no own energy): 7 (15.2%)
Total number of final clusters: 21
```

The **source** records where τ came from: **knee** in the first pass, and **user** here, because `--th=opt` passed on the value the first pass had found. The summary then lists every cluster with its members sorted by energy; the first member is the representative:

```
Cluster 1 (4 configurations):
Files:
- motif_21_opt.out (Gibbs Energy: -379.756973 Hartree (-238301.10 kcal/mol,
-10333.71 eV))
- motif_04_opt.out (Gibbs Energy: -379.756966 Hartree (-238301.09 kcal/mol,
-10333.71 eV))
- motif_01_opt.out (Gibbs Energy: -379.756317 Hartree (-238300.69 kcal/mol,
-10333.70 eV))
- motif_20_opt.out (Gibbs Energy: -379.756315 Hartree (-238300.69 kcal/mol,
-10333.70 eV))

Cluster 2 (3 configurations):
Files:
- motif_02_opt.out (Gibbs Energy: -379.748280 Hartree (-238295.64 kcal/mol,
-10333.48 eV))
- motif_15_opt.out (Gibbs Energy: N/A)
- motif_24_opt.out (Gibbs Energy: N/A)
```

Gibbs Energy: N/A marks a structure that has no Gibbs energy of its own, because its optimization did not converge, but that matched the cluster on the reduced feature vector. It is counted as redundant and never becomes a representative. The first pass reports **Electronic Energy** instead, since the xTB optimization has no frequencies.

11.4.4 Error Handling and Diagnostics

COSMIC checks every output for three kinds of problem: **execution failures** (no normal termination), **non-converged geometries** (the cycle limit was reached) and **imaginary frequencies** (saddle points, higher order stationary points or soft modes). A problematic structure that matches a cluster of true minima is already represented and is counted as redundant. One that matches nothing is **critical**: it may be a missing motif. The redo loop retries critical structures (Section 10.5.2); those still critical after the last attempt are exported to **skipped_structures/** for manual review.

The formic acid run ended with no critical structures, so it has no such folder. The rigorous glycolaldehyde–water run, which ended at 6.1% critical after three redo attempts, shows the layout:

```
cosmic_2/skipped_structures/
├─ clustered_with_minima/.....imaginary frequencies, but represented
├─ critical_non_converged/
│  └─ motif_22_opt.out | .xyz
│  └─ motif_39_opt.out | .xyz
│  └─ motif_59_opt.out | .xyz
│  └─ non_converged_summary.txt
├─ need_recalculation/.....imaginary frequencies, unmatched
│  └─ motif_66_opt.out | .xyz | .mol
└─ skipped_summary.txt
```

skipped_summary.txt gives the statistics and a recommendation:

```
Total structures with imaginary frequencies: 1
- Clustered with true minima (can be ignored): 0
- Not clustered structures (need review): 1
Percentage of structures needing review: 100.0%
Impact of critical structures on total dataset: 1/66 configurations (1.5%)
Assessment: Low impact (<10%)
Small fraction of dataset. Recalculation recommended but not critical.
```

Structures in `need_recalculation/` may be transition states or missing minima and should be reoptimized by hand, for example with a tighter method or starting from a displaced geometry. Those in `critical_non_converged/` reached the cycle limit on every attempt; a larger `maxiter` or a different optimizer setting is usually the fix.

11.4.5 Generated Cluster Data

Two folders give the detail behind each cluster:

1. `extracted_clusters/`: one subfolder per cluster, named `cluster_<id>_<n>` for a cluster with n members (`cluster_<id>` for a single member). It holds each member's geometry and a combined `.xyz/.mol` file for superposing them.
2. `extracted_data/`: one `cluster_<id>_<n>.dat` per cluster, with the statistics of its members.

The `.dat` file for cluster 2 of the final pass shows how similar each member is to the representative, which features were used, and how much each one varies within the cluster:

```
Cluster (represented by: cluster_2_3) (3 configurations)

- motif_02_opt.out
- motif_15_opt.out
- motif_24_opt.out

Pearson Similarity to Cluster Representative
-----
Similarity floor: 98.5%

Per-member similarity to representative:
motif_02_opt.out : d = 0.000   r = 1.000   similarity = 100.0 % (
representative)
motif_15_opt.out : d = 0.025   r = 1.000   similarity = 100.0 %
motif_24_opt.out : d = 0.509   r = 0.988   similarity = 98.8 %

Dynamic reduced feature vector.
Features not used: Dipole Moment, First Vibrational Frequency, Gibbs Free Energy,
Last Vibrational Frequency

(...)
```

Here the frequency and Gibbs energy features are dropped because two members never converged, which is exactly the reduced vector comparison that assigned them to this cluster. The file continues with the deviation of each feature within the cluster (Max–Min/|Mean|, 2.46% for

the HOMO–LUMO gap here), the weights and absolute tolerances applied, then the electronic and molecular descriptors, the vibrational frequency summary, the hydrogen bond analysis, a pairwise heavy atom RMSD table and the final geometry of every member.

11.4.6 Boltzmann Distribution

After the last COSMIC pass, `boltzmann_distribution.txt` (also copied to the project root) ranks the unique motifs by Gibbs free energy and gives each one its Boltzmann population, by default at 298.15 K. The `-T` flag changes the temperature of the ranking, but the Gibbs energies come from the QM frequency calculation and must be recomputed at that temperature for the result to be consistent. The file opens with the reference (lowest) configuration and then lists every motif:

```
Reference Configuration:
  Structure: motif_21_opt from cluster_1
  Reference Energy (Emin): -379.756973 Hartree (-238301.10 kcal/mol, -10333.71 eV)
  Temperature (T): 298.15 K

cluster_1 (umotif_01)
  From structure: motif_21_opt
  Gibbs Energy: -379.756973 Hartree (-238301.10 kcal/mol, -10333.71 eV)
  Relative Gibbs Energy ( $\Delta G$ ): 0.000000 Hartree (0.00 kcal/mol, 0.00 eV)
  Population: 99.99 %

cluster_2 (umotif_02)
  From structure: motif_02_opt
  Gibbs Energy: -379.748280 Hartree (-238295.64 kcal/mol, -10333.48 eV)
  Relative Gibbs Energy ( $\Delta G$ ): 0.008694 Hartree (5.46 kcal/mol, 0.24 eV)
  Population: 0.01 %
```

The `umotif_NN` numbering follows this ranking, and `final_ensemble.xyz` stores the motifs in the same order, so its first frame is the putative global minimum (Section 10.3.4).

12 Results and Validation

The COSMIC–ASCEC protocols were validated on four systems: the open shell lithium cluster $^2\text{Li}_5$, the water hexamer $(\text{H}_2\text{O})_6$, the formic acid dimer $(\text{H}_2\text{CO}_2)_2$, and glycolaldehyde with two water molecules $(\text{H}_2\text{O})_2\text{C}_2\text{H}_4\text{O}_2$. Table 12 summarizes the runs. In every case the only user input was the `.asc` file, and all calculations ran on a single local workstation.

Reference data. The `.asc` inputs, annealing outputs, Cartesian coordinates and the quantum chemistry input and output files of every stage are deposited in Zenodo (DOI: [10.5281/zenodo.20723683](https://doi.org/10.5281/zenodo.20723683)). The same runs are the source of every file excerpt in Section 11, so a new installation can be checked against them.

Cost. Preliminary runs spend most of their time in the annealing, evaluating thousands of semiempirical energies; none took more than about three minutes. In rigorous runs, over 99% of the wall time is spent in the DFT geometry refinement, and in ultimate runs the coupled cluster energies add most of the rest (Figure 17). The search is cheap because the expensive stages only see the structures COSMIC keeps. For the same reason, adding replicas is cheap: at least five are recommended, although the right number should be checked by a convergence test for each system.

System	Mode	Rep.	Energy evals	Candidates	Prelim.	Final	Wall time ^b
² Li ₅	Preliminary	1	3936	41	6	–	00:00:02:46
	Ultimate	3	9152	192	12	4	00:04:13:57
(H ₂ O) ₆	Preliminary	1	3254	78	19	–	00:00:03:07
	Rigorous	3	4683	215	39	22	01:03:26:58
	Ultimate	3	5111	259	52	26	04:14:04:22
(H ₂ CO ₂) ₂	Preliminary	1	1530	101	23	–	00:00:01:16
	Rigorous	5	4066	574	46	21	00:10:42:34
(H ₂ O) ₂ C ₂ H ₄ O ₂	Preliminary	1	657	108	40	–	00:00:00:55
	Rigorous	3	2566	269	66	44, 28 ^a	01:11:00:12

Table 12: Benchmark runs. *Candidates*: configurations accepted by the annealing; *Prelim.*: motifs after the first COSMIC screening; *Final*: unique motifs after refinement. ^aKnee threshold ($\tau = 0.29$) and empirical threshold ($\tau = 2.0$), respectively. ^bdd:hh:mm:ss, as in Figure 17.

12.1 Screening Plain Coordinates

COSMIC does not depend on ASCEC or on any quantum chemistry output: a folder of `.xyz` files, or a single multiframe `.xyz` such as a trajectory or a concatenated ensemble, is enough. From coordinates alone COSMIC builds 8 of the 15 components of the feature vector (Section 6.3.5): the nuclear repulsion, the three rotational constants and the four hydrogen bond descriptors. `--sp` adds four more (electronic energy, HOMO, HOMO–LUMO gap and dipole) through one GFN2-xTB single point per structure, with the charge and unpaired electrons given by `--charge` and `--uhf`. The Gibbs energy and the vibrational frequencies need a frequency calculation and never come from coordinates.

```
cosmic xyz_pool/ -j4          # geometry only
cosmic xyz_pool/ --sp -j4     # plus xTB single points
```

Two preliminary runs of Table 12 were screened this way, from their xTB optimized geometries alone, and compared with the clustering of the same structures from their xTB outputs (Table 13).

System	Structures	From <code>.xyz</code>		From xTB outputs	
		Components	Motifs	Components	Motifs
(H ₂ O) ₆	78	8	22	12	19
² Li ₅	41	4	6	8	6

Table 13: COSMIC on plain coordinates and on the xTB outputs of the same structures.

For the water hexamer, the geometric and hydrogen bond descriptors alone gave 22 motifs, a few more redundancies than the 19 found with the electronic descriptors. Li₅ is the extreme case: with no hydrogen bonds and no electronic structure, only the rotational constants and the nuclear repulsion remain, and those four components still gave the same six motifs. Two differences from a QM run follow from the missing energies:

- **No ranking.** Energies written in the comment line of an external `.xyz` are of unknown origin and are not used. Without `--sp` the structures are not sorted, and the representative of each family is the member closest to the family mean.

- **Composition check.** Every structure must have the same elements and counts; otherwise COSMIC stops and names the files that differ.

A solvated molecular dynamics trajectory is first reduced to the solute and its solvation shell (`--shell`, `--nearest`; see `cosmic --help`).

12.2 Clustering Methodologies: Topology vs. Geometry

COSMIC compares structures through the physicochemical feature vector (Section 6.3); the usual alternative is the root mean square deviation (RMSD) of the Cartesian coordinates. The two differ in one decisive respect: RMSD compares atoms by their index, while every component of the feature vector is invariant to how the atoms are numbered.

12.2.1 Invariance to Atom Indexing

Figure 26 shows the same $\text{H}_2\text{O} \cdots \text{H}_2\text{AsO}_4^-$ cluster three times, with only the atom labels shuffled: the two water hydrogens are swapped, and then the oxygens and hydroxyl hydrogens of the anion. The geometry is identical. RMSD, computed atom by atom, classifies the three as different structures; COSMIC classifies them as the same. In a stochastic search, where equivalent atoms are exchanged freely, this is the difference between one motif and several copies of it.

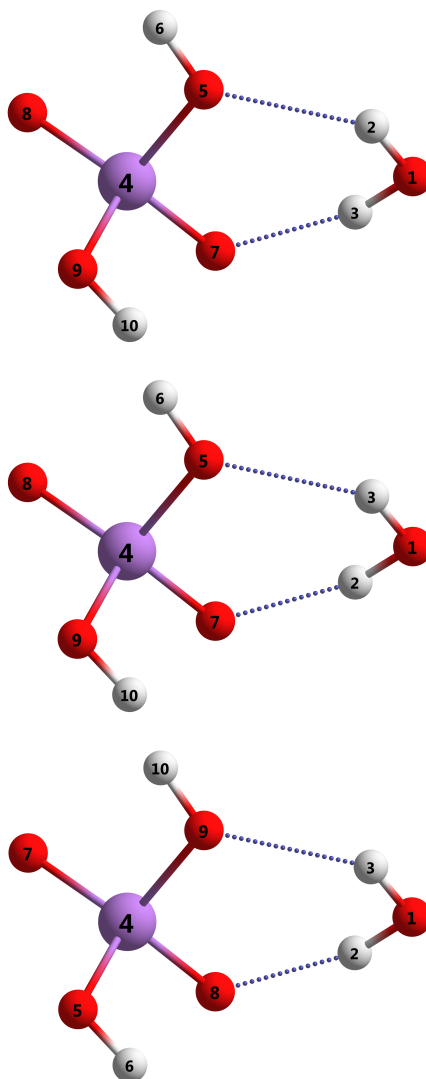


Figure 26: One $\text{H}_2\text{O}\cdots\text{H}_2\text{AsO}_4^-$ geometry with shuffled atom labels. RMSD classifies these structures as different; COSMIC classifies them as the same.

12.2.2 Comparison with Existing Methods

Table 14 applies COSMIC to water hexamer ensembles produced by other programs, both to their original pools and to the pools after each program's own redundancy filter. The *COSMIC families* columns give the number of structures left when COSMIC groups the pool with the physicochemical vector, or with an RMSD cut of 0.125 Å or 1.0 Å.

Program	Ensemble	n	COSMIC families		
			Phys. vector	RMSD 0.125 Å	RMSD 1.0 Å
CREST 3.0.2	Original pool	276	63	276	179
	After CREGEN	74	19	74	62
CREST 3.1.0 (iRMSD)	Original pool	346	82	345	224
	After CREGEN	96	20	95	82
GOAT (ORCA 6.1.1)	Original pool	820	67	736	468
	After GOAT filter	187	26	184	156
OGOLEM 2021.07	Pool	100	22	99	89
COSMIC-ASCEC	Own sampling	57	14	57	47

Table 14: Water hexamer: structures remaining after grouping each ensemble with COSMIC’s physicochemical vector or with RMSD.

The original CREST 3.0.2 pool holds 276 structures. The physicochemical vector reduces it to 63, removing over two hundred redundancies, while RMSD at 0.125 Å removes none. CREGEN reduces the same pool to 74 structures, and COSMIC reduces those 74 further to 19 motifs. The pattern is the same for every program: RMSD at either cut leaves most of the redundancies that the physicochemical vector removes.

12.2.3 Quantifying a Visual Redundancy

Two structures that look alike are not necessarily the same. Three of the six Li₅ preliminary motifs (S₁, S₂ and S₅) appear redundant at first sight, but in S₁ the external Li atom lies in a plane bisecting the pyramid, which S₂ and S₅ lack, and every atom–atom distance differs among the three. `--compare` measures the difference directly on the output files, without a folder or a dendrogram:

```
cosmic --compare s1.out s2.out s5.out
```

The 8 component vectors of the xTB optimized structures differ by 28% in the dipole moment, 15% in the HOMO–LUMO gap and up to 10% in the rotational constants: three different structures. The comparison is written to `clustering_summary.txt` and a single `cluster_*.dat` with the per feature deviations, which replaces a visual judgment with a number.

12.2.4 When to Use RMSD

RMSD remains available as a secondary, geometric split inside COSMIC (`--rmsd`, Section 9.4). It runs after the topological clustering and separates members of a family that share their descriptors but differ in shape. Use it when the spatial arrangement itself matters, knowing that it is sensitive to atom ordering and will split families that are really the same motif. For general PES exploration the default, clustering on the physicochemical vector alone, is the recommended choice.

13 Command Reference

13.1 Cheat Sheet

The commands below cover almost every run. Most users only need the last block — the automated protocol drives every stage from a single `.asc` file.

```
# --- Sampling -----
ascec input.asc                # annealing only (no protocol block)
ascec input.asc r5 --box20 --concurrent=5  # 5 replicas, 20% packing
ascec input.asc box            # box / packing analysis only

# --- Build QM decks from the results -----
ascec opt template.inp launcher.sh  # optimization inputs (+launcher)
ascec ref template.inp launcher.sh  # refinement inputs (+launcher)
ascec update new_template.inp       # rewrite existing decks, keep coordinates
ascec launcher                     # merge all launcher_ascec.sh into one
ascec merge result                  # pool result_*.xyz into combined_results.
xyz

# --- Collect & cluster -----
ascec sort                        # rank optimized structures
cosmic -j4                       # cluster (auto threshold, 4 cores)
cosmic --rmsd=1 -j4              # + geometric split at 1.0 Å
cosmic --th=opt -j4              # refinement stage: reuse postopt tau
cosmic --partialweights -j4      # preliminary xTB / semiempirical

# --- Reproducibility (seeds) -----
COSMIC_ASCEC_SEED=42 ascec input.asc      # one seed, all replicas
COSMIC_ASCEC_SEEDS=1,2,3 ascec input.asc r3  # one seed per replica

# --- Housekeeping -----
ascec status                     # view, attach to or kill running jobs
ascec diagram                   # energy plots from the tvse_*.dat files
ascec cleanup                   # remove temp calc/cosmic folders
ascec input.asc --maxprint       # keep all intermediate files
ascec input.asc --review         # rehearse the protocol on the putative
global minimum

# --- One shot automated protocol (recommended) -----
ascec input.asc                 # run the protocol embedded in the .asc
ascec input.asc after 1131777   # queue behind a running job (PID)
ascec input.asc protocol 3      # restart from stage 3 (deletes 3 onwards)
ascec input.asc protocol 3 -i   # continue stage 3, keeping its files
ascec input.asc exclude ref 3,7-9 # drop motifs 3, 7-9 at a breakpoint
```

13.2 Automated Workflow (Protocol)

Controls the execution of the multistage pipeline (Section 10).

Command Format	Description
<code>ascec input.asc</code>	Runs the protocol embedded in the file, or resumes it from the first unfinished stage. <code>ascec input.asc protocol</code> is equivalent.
<code>... protocol N</code>	Restart. Deletes the directories and cache entries of stage N and every later stage, then reruns from N (Section 10.4).
<code>... protocol N -i</code>	Continue. Marks stage N as incomplete without deleting anything, so only the missing calculations run.
<code>... protocol opt</code>	Stages can be named instead of numbered: <code>opt</code> , <code>opt2</code> , ..., <code>ref</code> , <code>ref2</code> , ...
<code>... after <PID></code>	Queues the run behind a running job; it starts when that PID finishes.
<code>--review</code>	Global modifier: carries only the putative global minimum through the expensive stages (Section 10.2).
<code>--maxprint</code>	Global modifier: keeps every intermediate file. The default, <code>miniprint</code> , cleans the stage folders at the end (Section 11.1).

13.3 Protocol Stage Flags

Written on an `opt` or `ref` line of the protocol; the redo flags control the COSMIC pass that follows that stage (Section 10.5). Launch failures are always retried automatically, up to 10 times.

Flag	Description
<code>--concurrent=N</code>	Concurrency. Number of QM jobs run in parallel in this stage (default 1).
<code>--redo=N</code>	Redo Limit. Maximum redo attempts for critical structures (default 3).
<code>--critical=N</code>	Critical Threshold. Maximum % of critical structures (skipped structures that match no cluster) accepted before a redo is triggered (default 0).
<code>--skipped=N</code>	Skipped Threshold. Replaces the critical criterion: maximum % of all skipped structures (imaginary frequencies or not converged).

13.4 COSMIC Analysis Module

Invoked via `ascec cosmic` or `cosmic`, this module handles clustering and Boltzmann analysis.

Flag	Description
--th, --threshold	COSMIC Threshold. Defines the clustering granularity. Default <code>auto</code> (knee detection, capped at $\tau = 2.0$; see Section 6.3.7); a float forces a fixed cut, lower values giving stricter clustering.
--rmsd	Geometric Validation. Enables RMSD checking in Ångströms (Default: 1.0 Å). Useful for distinguishing conformers with similar energies but different shapes.
-j, --cores	Parallelization. Number of CPU cores to use for parsing and matrix calculation (Default: Autodetect).
-T, --temperature	Boltzmann Temp. Temperature in Kelvin for Gibbs Free Energy ranking (Default: 298.15 K).
--weights	Custom Weights. Define specific feature importance (e.g., '(energy=0.8)(dipole=0.15)').
--group-hb	H-Bond Grouping. Groups structures by hydrogen bond count before clustering. Produces separate dendrograms per HB family (<code>dendrogram_H{N}.png</code>). Useful for visual inspection of specific interaction networks.
--sp	Single points. For <code>.xyz</code> input: one GFN2-xTB single point per structure adds energy, HOMO, gap and dipole to the geometric vector; <code>--charge</code> and <code>--uhf</code> set the charge and unpaired electrons (Section 12.1).
--compare	Direct comparison. Compares two or more files and reports the per feature deviations, with no dendrogram.
--reprocess-files	Ignores the cached <code>.pkl</code> data and forces a reread of all output.

14 AI Acknowledgement and Reproducibility

The development and documentation of the ASCEC protocol involved the use of large language models (LLMs) to improve code optimization, data synthesis, and technical writing. Specifically, Claude Opus 4.5 [15] and Gemini 3.0 [16] were used to streamline the programming process and refine the algorithmic logic.

In accordance with ethical research standards, all AI assisted results underwent rigorous human verification to ensure scientific accuracy and reproducibility. We encourage users who adapt this methodology to maintain transparency regarding the use of computational assistants and to verify all results.

15 Glossary

These terms recur throughout the manual and in the program’s output.

Configuration vs conformation A *configuration* is an arrangement of whole molecules relative to one another; a *conformation* is the internal shape of one molecule, set by its dihedral angles. ASCEC samples both.

Motif One representative structure standing for a family of structures COSMIC judged equivalent. Reducing hundreds of candidates to a handful of motifs is what makes the expensive stages affordable.

Replica An independent annealing run of the same system with a different random seed. Written `rN` in a protocol, so `r3` means three replicas. More replicas cover the surface better.

Concurrent How many quantum chemistry jobs run at the same time (`--concurrent=N`). This is about throughput, not sampling, and is bounded by your cores and memory.

Threshold τ The distance at which the clustering tree is cut. A smaller τ splits structures into more, finer families; a larger one merges them. It can be chosen automatically.

Critical structure A refined structure that came back incomplete or with imaginary frequencies, so it is not a true minimum. COSMIC reports the percentage of these.

Redo An automatic repeat of a stage for the critical structures, nudging them off the saddle point and reoptimizing, until none are left or the attempt limit is reached.

miniprint / maxprint How much is kept on disk. Under **miniprint** (the default) only the final ensemble's individual quantum chemistry files survive; `--maxprint` keeps everything from every stage.

review run A protocol launched with `--review`: every stage executes, but only the lowest energy annealing structure (the putative global minimum) enters the per structure stages. Used to validate an expensive protocol before running it in full.

16 References

- [1] F. Neese. "Software Update: The ORCA Program System—Version 6.0". In: *WIREs Comput. Mol. Sci.* 15.2 (2025). DOI: [10.1002/wcms.70019](https://doi.org/10.1002/wcms.70019).
- [2] C. Bannwarth, S. Ehlert, and S. Grimme. "GFN2-xTB—An Accurate and Broadly Parametrized Self-Consistent Tight-Binding Quantum Chemical Method with Multipole Electrostatics and Density-Dependent Dispersion Contributions". In: *J. Chem. Theory Comput.* 15.3 (2019). DOI: [10.1021/acs.jctc.8b01176](https://doi.org/10.1021/acs.jctc.8b01176).
- [3] S. Kim et al. "An Update on PUG-REST: RESTful Interface for Programmatic Access to PubChem". In: *Nucleic Acids Res.* 46.W1 (2018). DOI: [10.1093/nar/gky294](https://doi.org/10.1093/nar/gky294).
- [4] N. Rego and D. Koes. "3Dmol.js: Molecular Visualization with WebGL". In: *Bioinformatics* 31.8 (2015). DOI: [10.1093/bioinformatics/btu829](https://doi.org/10.1093/bioinformatics/btu829). PMID: [25505090](https://pubmed.ncbi.nlm.nih.gov/25505090/).
- [5] G. Hincapié et al. "Structural Studies of the Water Hexamer". In: *J. Phys. Chem. A* 114.29 (2010). DOI: [10.1021/jp103683m](https://doi.org/10.1021/jp103683m).
- [6] P. J. van Laarhoven and E. H. Aarts. *Simulated Annealing: Theory and Applications*. Springer Science & Business Media, 1987.
- [7] N. Metropolis et al. "Equation of State Calculations by Fast Computing Machines". In: *J. Chem. Phys.* 21.6 (1953). DOI: [10.1063/1.1699114](https://doi.org/10.1063/1.1699114).
- [8] J. F. Pérez, C. Z. Hadad, and A. Restrepo. "Structural Studies of the Water Tetramer". In: *Int. J. Quantum Chem.* 108.10 (2008). DOI: [10.1002/qua.21615](https://doi.org/10.1002/qua.21615).
- [9] T. M. Abramyan et al. "Cluster Analysis of Molecular Simulation Trajectories for Systems Where Both Conformation and Orientation of the Sampled States Are Important". In: *J. Comput. Chem.* 37.21 (2016). DOI: [10.1002/jcc.24416](https://doi.org/10.1002/jcc.24416). PMID: [27292100](https://pubmed.ncbi.nlm.nih.gov/27292100/).
- [10] W. Kabsch. "A Solution for the Best Rotation to Relate Two Sets of Vectors". In: *Acta Crystallogr. Sect. A* 32.5 (1976). DOI: [10.1107/S0567739476001873](https://doi.org/10.1107/S0567739476001873).
- [11] R. R. Sokal and C. D. Michener. "A Statistical Method for Evaluating Systematic Relationships". In: *Univ. Kans. Sci. Bull.* 38.22 (1958).
- [12] V. Satopaa et al. "Finding a "Kneedle" in a Haystack: Detecting Knee Points in System Behavior". In: *Proc. 2011 31st Int. Conf. Distrib. Comput. Syst. Workshop. ICDCSW '11. USA: IEEE Computer Society, 2011*. DOI: [10.1109/ICDCSW.2011.20](https://doi.org/10.1109/ICDCSW.2011.20).

- [13] R. Mojena. “Hierarchical Grouping Methods and Stopping Rules: An Evaluation*”. In: *Comput J* 20.4 (1977). DOI: [10.1093/comjnl/20.4.359](https://doi.org/10.1093/comjnl/20.4.359).
- [14] P. Farfán et al. “Dimers of Formic Acid: Structures, Stability, and Double Proton Transfer”. In: *J. Chem. Phys.* 147.4 (2017). DOI: [10.1063/1.4985880](https://doi.org/10.1063/1.4985880).
- [15] Anthropic. *System Card: Claude Opus 4.5*. <https://www.anthropic.com/claude-opus-4-5-system-card>. 2025.
- [16] Google DeepMind. *Gemini 3 Deep Think: Advancing Science, Research and Engineering*. <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-deep-think/>. 2026.